

COMPLETE PROGRAMMING

Upper Secondary / Terminale - Scientific Stream

Programming Concepts - Python Examples - Algorithms

30 Complete Lessons - Practice Activities

Revision Tests - Answer Key - Reference Tables

A structured 80-page programming coursebook for classroom teaching, independent study, practical coding, and exam preparation.

Focus language: Python-style examples with general programming principles.

Includes algorithms, flowcharts, control structures, data structures, functions, files, debugging, OOP, searching, sorting, complexity and SQL.

PROGRAMMING SCIENTIFICS SERIES

GETTING STARTED

How to Use This Book

A practical roadmap for learning programming step by step.

What You Will Learn

- Programming foundations from algorithms and data types to object-oriented design and SQL.
- Accurate coding habits for school, projects, exams, and independent practice.
- Debugging and testing strategies that help you find, explain, and correct errors.
- Programming in context through examples, mini-tables, guided practice, revision tests, and answer keys.

Recommended Study Method

1. Read the lesson objective before studying the concepts.
2. Type the model code yourself instead of only reading it.
3. Complete guided practice without looking at the answers.
4. Trace code by hand when you do not understand the output.
5. Review difficult lessons after 24 hours and again after one week.

Programming Checklist

- Problem: Are the inputs, outputs, and rules clear?
- Logic: Does the algorithm cover all required cases?
- Variables: Are names and data types suitable?
- Control flow: Are conditions and loops correct and terminating?
- Functions: Are tasks decomposed into clear reusable units?
- Testing: Have normal, boundary, and invalid cases been checked?
- Style: Is the code readable, documented, and consistently formatted?

Book Organization

- Lessons 1-10 build the foundations: problem solving, algorithms, variables, expressions, input/output, selection, and loops.
- Lessons 11-20 develop strings, collections, functions, recursion, modules, files, debugging, testing, and OOP.
- Lessons 21-30 complete the course with classes, inheritance, algorithms, complexity, data representation, stacks, queues, databases, and a mini-project.

Suggested 6 Week Study Schedule

- Week 1: Lessons 1-5 - problem solving, algorithms, flowcharts, variables, operators.
- Week 2: Lessons 6-10 - input/output, selection, loops, nested control.
- Week 3: Lessons 11-15 - strings, collections, functions, parameters, recursion.
- Week 4: Lessons 16-20 - modules, files, debugging, testing, OOP concepts.
- Week 5: Lessons 21-25 - classes, inheritance, searching, sorting, complexity.
- Week 6: Lessons 26-30 + revision tests - data representation, data structures, databases, project.

REFERENCE

Table of Contents

Lessons 1-15

Lesson	Topic	Pages
1	Programming Fundamentals and Problem Solving	5-6
2	Algorithms and Pseudocode	7-8
3	Flowcharts and Program Design	9-10
4	Variables, Constants and Data Types	11-12
5	Operators and Expressions	13-14
6	Input, Output and Type Conversion	15-16
7	Conditional Statements	17-18
8	While Loops	19-20
9	For Loops and Range	21-22
10	Strings and Text Processing	23-24
11	Lists and Arrays	25-26
12	Tuples, Sets and Dictionaries	27-28
13	Functions	29-30
14	Parameters, Return Values and Scope	31-32
15	Recursion	33-34

Learning Map

- Lessons 1-5: problem solving, algorithms, flowcharts, variables, and operators.
- Lessons 6-10: input/output, conditionals, while/for loops, and nested control.
- Lessons 11-15: strings, lists, dictionaries, functions, scope, and recursion.

Milestones

- I can design an algorithm before coding.
- I can write readable Python programs using variables, conditions, loops, and functions.
- I can test and debug programs systematically.
- I can explain common algorithms, data structures, and object-oriented concepts.

REFERENCE

Table of Contents

Lessons 16-30, Revision and Reference

Lesson	Topic	Pages
16	Modules and Libraries	35-36
17	File Handling	37-38
18	Exceptions and Debugging	39-40
19	Testing and Test Cases	41-42
20	Object-Oriented Programming Concepts	43-44
21	Classes and Objects in Python	45-46
22	Inheritance and Polymorphism	47-48
23	Searching Algorithms	49-50
24	Sorting Algorithms	51-52
25	Algorithm Complexity and Efficiency	53-54
26	Boolean Logic and Binary Numbers	55-56
27	Data Representation and Memory	57-58
28	Stacks and Queues	59-60
29	Databases and SQL Basics	61-62
30	Mini-Project Design and Programming Review	63-64
Revision	Comprehensive Review Tests	65-72
Answers	Answer Key	73-78
Reference	Python & Programming Reference	79-80

Learning Map

- Lessons 16-20: modules, files, exceptions, testing, and OOP concepts.
- Lessons 21-25: classes, inheritance, searching, sorting, and complexity.
- Lessons 26-30: Boolean/binary, data representation, stacks/queues, SQL, and project review.

Milestones

- I can use classes and objects appropriately.
- I can compare searching and sorting strategies.
- I can explain basic complexity, Boolean logic, and data representation.
- I can plan, test, document, and evaluate a complete mini-project.

LESSON 01 - CORE PROGRAMMING

Programming Fundamentals and Problem Solving

Concepts, syntax, and model examples

Learning Objectives

- Explain what a program, algorithm, and programming language are.
- Describe the stages of solving a computational problem.
- Distinguish syntax, semantics, and logic errors.

Concept	Explanation	Example
Program	A sequence of instructions executed by a computer.	A script that calculates a student average.
Algorithm	A finite, ordered method for solving a problem.	Input marks -> compute sum -> divide by count -> display average.
Language	A formal system for writing executable instructions.	Python, Java, C++, JavaScript.
Problem-solving cycle	Analyse, design, code, test, debug, document, improve.	Plan first, then implement and test.

Key Rules

- A good program solves a clearly defined problem before it uses advanced syntax.
- Algorithms should be finite, unambiguous, and ordered.
- Separate syntax errors from logic errors during debugging.

Study Tip

Type each model example yourself. Change one value or condition and predict the result before running it.

Extra Model Code / Sentences

1. Problem: calculate the area of a rectangle.
2. Algorithm: read length and width; area = length * width; print area.
3. Python: area = length * width
4. Output should be checked with known test values.

Worked Analysis Routine

Step 1: Identify the inputs, outputs, and target operation.
 Step 2: Trace variables and conditions in execution order.
 Step 3: Explain why the chosen syntax or structure matches the intended logic.
 Step 4: Modify one input or rule and predict how the output changes.

Your Example

Write one accurate example or short code fragment and explain the target programming idea.

LESSON 01 - PRACTICE

Programming Fundamentals and Problem Solving

Common errors, guided practice, and mastery check

Common error	Better form	Why?
Start coding before understanding the problem.	Write inputs, outputs, and steps first.	Planning reduces logic errors.
Assume a program is correct because it runs.	Test with normal, boundary, and invalid cases.	Running without crashing does not prove correctness.

Guided Practice

1. Define programming in one sentence.
2. List the six main stages of the problem-solving cycle.
3. Give one example of an input and one output for a calculator program.
4. Explain the difference between syntax and logic errors.
5. Write an algorithm to convert minutes to seconds.
6. State two qualities of a good algorithm.
7. Give one reason why testing is necessary.
8. Describe a simple real-life task that can be automated.

Mastery Challenge

- Write two original examples demonstrating the main concept of this lesson.
- Trace or explain the expected output/result in one short paragraph.
- Check syntax, indentation, names, and edge cases before comparing with model answers.

Quick Answers

1. Writing instructions for a computer to solve problems.
2. Analyse, design, code, test, debug, document/improve.
3. Example: input 4 and 5; output 9.
4. Syntax breaks language rules; logic gives wrong results.
5. Read minutes; seconds=minutes*60; display seconds.
6. Finite and unambiguous (also ordered/effective).
7. To detect errors and verify expected results.
8. Answers vary.

Independent Practice

- A. Write three short code examples using Programming Fundamentals and Problem Solving.
- B. Change one example so it handles a different input or condition.
- C. Write one deliberately incorrect example, then correct it and explain the error.
- D. Use the concept in a short real-life programming task.

Exam Strategy

- Read the complete problem before choosing a structure.
- Trace small examples by hand when output is uncertain.
- When two solutions work, explain differences in clarity, efficiency, or robustness.

Notes & Corrections

Record one mistake you made, the correction, and the programming rule or idea.

LESSON 02 - CORE PROGRAMMING

Algorithms and Pseudocode

Concepts, syntax, and model examples

Learning Objectives

- Write algorithms using sequence, selection, and repetition.
- Use clear pseudocode independent of a specific language.
- Trace an algorithm manually using sample data.

Concept	Explanation	Example
Sequence	Instructions executed in order.	READ a; READ b; sum <- a+b; PRINT sum
Selection	Choose a path based on a condition.	IF mark >= 50 THEN PRINT "Pass"
Repetition	Repeat instructions while or for a condition.	FOR i <- 1 TO 10 DO PRINT i
Trace table	Records variable values step by step.	Track i, total, and output for each loop iteration.

Key Rules

- Use meaningful variable names in pseudocode.
- Indent blocks to make control structure clear.
- Trace with small sample values before coding.

Study Tip

Type each model example yourself. Change one value or condition and predict the result before running it.

Extra Model Code / Sentences

```
1. READ radius
2. area <- 3.14159 * radius * radius
3. IF area > 100 THEN PRINT "Large" ELSE PRINT "Small"
4. END IF
```

Worked Analysis Routine

- Step 1: Identify the inputs, outputs, and target operation.
- Step 2: Trace variables and conditions in execution order.
- Step 3: Explain why the chosen syntax or structure matches the intended logic.
- Step 4: Modify one input or rule and predict how the output changes.

Your Example

Write one accurate example or short code fragment and explain the target programming idea.

LESSON 02 - PRACTICE

Algorithms and Pseudocode

Common errors, guided practice, and mastery check

Common error	Better form	Why?
Use language-specific syntax in every pseudocode line.	Use generic words such as READ, PRINT, IF, WHILE.	Pseudocode should express logic, not exact syntax.
Skip indentation.	Indent statements inside blocks.	Indentation makes structure visible.

Guided Practice

1. Write pseudocode to add three numbers.
2. Write pseudocode to test whether a number is positive.
3. Write a FOR loop that prints 1 to 5.
4. What is a trace table used for?
5. Identify sequence, selection, and repetition in a school attendance system.
6. Trace total after adding 2, 4, 6.
7. Rewrite a long algorithm using meaningful names.
8. Explain why pseudocode is useful before coding.

Mastery Challenge

- Write two original examples demonstrating the main concept of this lesson.
- Trace or explain the expected output/result in one short paragraph.
- Check syntax, indentation, names, and edge cases before comparing with model answers.

Quick Answers

1. READ a,b,c; total<-a+b+c; PRINT total.
2. IF n>0 THEN PRINT "Positive".
3. FOR i<-1 TO 5 DO PRINT i.
4. To follow variable values and logic step by step.
5. Answers vary.
6. 2, then 6, then 12.
7. Answers vary.
8. It helps design and check logic independently of a language.

Independent Practice

- A. Write three short code examples using Algorithms and Pseudocode.
- B. Change one example so it handles a different input or condition.
- C. Write one deliberately incorrect example, then correct it and explain the error.
- D. Use the concept in a short real-life programming task.

Exam Strategy

- Read the complete problem before choosing a structure.
- Trace small examples by hand when output is uncertain.
- When two solutions work, explain differences in clarity, efficiency, or robustness.

Notes & Corrections

Record one mistake you made, the correction, and the programming rule or idea.

LESSON 03 - CORE PROGRAMMING

Flowcharts and Program Design

Concepts, syntax, and model examples

Learning Objectives

- Recognize standard flowchart symbols.
- Convert a simple algorithm into a flowchart.
- Follow program flow from start to end.

Concept	Explanation	Example
Terminator	Shows start or end.	START / END
Process	Represents a calculation or assignment.	total <- price * quantity
Input/Output	Represents reading or displaying data.	READ age / PRINT result
Decision	Branches according to a condition.	age >= 18? Yes / No

Key Rules

- Every flowchart should have a clear start and end.
- Decision branches must be labelled, usually Yes/No or True/False.
- Arrows should show one unambiguous direction of control.

Study Tip

Type each model example yourself. Change one value or condition and predict the result before running it.

Extra Model Code / Sentences

1. START
2. READ score
3. Is score >= 50? -> Yes: PRINT Pass; No: PRINT Fail
4. END

Worked Analysis Routine

Step 1: Identify the inputs, outputs, and target operation.
 Step 2: Trace variables and conditions in execution order.
 Step 3: Explain why the chosen syntax or structure matches the intended logic.
 Step 4: Modify one input or rule and predict how the output changes.

Your Example

Write one accurate example or short code fragment and explain the target programming idea.

LESSON 03 - PRACTICE

Flowcharts and Program Design

Common errors, guided practice, and mastery check

Common error	Better form	Why?
Use a rectangle for a decision.	Use a diamond for a condition.	Standard symbols improve readability.
Leave decision branches unlabeled.	Label branches Yes/No or True/False.	The path must be clear.

Guided Practice

1. Name the symbol used for a process.
2. Name the symbol used for input/output.
3. Draw or describe a flowchart for checking voting age.
4. Convert READ a,b; PRINT a+b into a flowchart.
5. Why are arrows necessary?
6. What should a decision node contain?
7. Give one advantage of flowcharts.
8. State one limitation of very large flowcharts.

Mastery Challenge

- Write two original examples demonstrating the main concept of this lesson.
- Trace or explain the expected output/result in one short paragraph.
- Check syntax, indentation, names, and edge cases before comparing with model answers.

Quick Answers

1. Rectangle.
2. Parallelogram.
3. START -> READ age -> age>=18? -> outputs -> END.
4. START -> READ a,b -> process sum -> PRINT sum -> END.
5. They show control flow.
6. A condition with labelled branches.
7. Visualizes program logic.
8. They can become complex and difficult to maintain.

Independent Practice

- A. Write three short code examples using Flowcharts and Program Design.
- B. Change one example so it handles a different input or condition.
- C. Write one deliberately incorrect example, then correct it and explain the error.
- D. Use the concept in a short real-life programming task.

Exam Strategy

- Read the complete problem before choosing a structure.
- Trace small examples by hand when output is uncertain.
- When two solutions work, explain differences in clarity, efficiency, or robustness.

Notes & Corrections

Record one mistake you made, the correction, and the programming rule or idea.

LESSON 04 - CORE PROGRAMMING

Variables, Constants and Data Types

Concepts, syntax, and model examples

Learning Objectives

- Create valid variables and assign values.
- Distinguish integer, float, string, and Boolean data.
- Choose suitable data types for program values.

Concept	Explanation	Example
Variable	A named storage location whose value may change.	score = 75
Constant	A value intended not to change during execution.	PI = 3.14159
Numeric types	Integers store whole numbers; floats store decimals.	age = 17; mass = 52.5
Text and Boolean	Strings store text; Booleans store True/False.	name = "Aina"; passed = True

Key Rules

- Use descriptive variable names such as total_price, not x when clarity matters.
- Do not start identifiers with a digit.
- Choose a data type that matches the meaning and required operations.

Study Tip

Type each model example yourself. Change one value or condition and predict the result before running it.

Extra Model Code / Sentences

```
1. name = "Miora"
2. age = 18
3. average = 14.75
4. is_registered = True
```

Worked Analysis Routine

Step 1: Identify the inputs, outputs, and target operation.
 Step 2: Trace variables and conditions in execution order.
 Step 3: Explain why the chosen syntax or structure matches the intended logic.
 Step 4: Modify one input or rule and predict how the output changes.

Your Example

Write one accurate example or short code fragment and explain the target programming idea.

LESSON 04 - PRACTICE

Variables, Constants and Data Types

Common errors, guided practice, and mastery check

Common error	Better form	Why?
2score = 10	score2 = 10	Identifiers cannot begin with a digit.
age = "18" when arithmetic is required	age = 18	Use a numeric type for numeric calculations.

Guided Practice

1. Classify 42, 3.5, "Hello", True by type.
2. Create a variable for student_name.
3. Create a constant for TAX_RATE.
4. Explain the difference between integer and float.
5. Which type should store a yes/no answer?
6. Correct: 1name = "Sara".
7. Why is "100" different from 100?
8. Give one situation where a constant is useful.

Mastery Challenge

- Write two original examples demonstrating the main concept of this lesson.
- Trace or explain the expected output/result in one short paragraph.
- Check syntax, indentation, names, and edge cases before comparing with model answers.

Quick Answers

1. int, float, string, bool.
2. student_name = "...".
3. TAX_RATE = 0.20 (example).
4. Integer is whole; float may include decimals.
5. Boolean.
6. name1 = "Sara" or name = "Sara".
7. One is text; the other is numeric.
8. PI, tax rate, conversion factor, etc.

Independent Practice

- A. Write three short code examples using Variables, Constants and Data Types.
- B. Change one example so it handles a different input or condition.
- C. Write one deliberately incorrect example, then correct it and explain the error.
- D. Use the concept in a short real-life programming task.

Exam Strategy

- Read the complete problem before choosing a structure.
- Trace small examples by hand when output is uncertain.
- When two solutions work, explain differences in clarity, efficiency, or robustness.

Notes & Corrections

Record one mistake you made, the correction, and the programming rule or idea.

LESSON 05 - CORE PROGRAMMING

Operators and Expressions

Concepts, syntax, and model examples

Learning Objectives

- Use arithmetic, comparison, logical, and assignment operators.
- Predict expression results using precedence.
- Build compound Boolean conditions.

Concept	Explanation	Example
Arithmetic	Perform mathematical operations.	+ - * / // % **
Comparison	Compare values and return Boolean results.	== != < <= > >=
Logical	Combine or invert Boolean expressions.	and, or, not
Assignment	Store or update a value.	= += -= *=

Key Rules

- Use parentheses when precedence may be unclear.
- Use == for comparison and = for assignment in Python.
- Modulo (%) is useful for divisibility and parity tests.

Study Tip

Type each model example yourself. Change one value or condition and predict the result before running it.

Extra Model Code / Sentences

```
1. total = 3 + 4 * 2 # 11
2. average = (a + b + c) / 3
3. is_even = n % 2 == 0
4. eligible = age >= 18 and registered
```

Worked Analysis Routine

- Step 1: Identify the inputs, outputs, and target operation.
- Step 2: Trace variables and conditions in execution order.
- Step 3: Explain why the chosen syntax or structure matches the intended logic.
- Step 4: Modify one input or rule and predict how the output changes.

Your Example

Write one accurate example or short code fragment and explain the target programming idea.

LESSON 05 - PRACTICE

Operators and Expressions

Common errors, guided practice, and mastery check

Common error	Better form	Why?
if x = 5:	if x == 5:	Comparison uses ==.
result = 10 / 2 + 3 without checking precedence	Use parentheses when needed.	Clear grouping prevents wrong assumptions.

Guided Practice

1. Evaluate $2 + 3 * 4$.

2. Evaluate $(2 + 3) * 4$.

3. What does `17 % 5` return?

4. Write a condition for age between 18 and 60.

5. Explain `==` versus `=`.

6. Write an expression for the square of x.

7. Use `+=` to add 5 to total.

8. Predict not $(3 > 2)$.

Mastery Challenge

- Write two original examples demonstrating the main concept of this lesson.
- Trace or explain the expected output/result in one short paragraph.
- Check syntax, indentation, names, and edge cases before comparing with model answers.

Quick Answers

1. 14.

2. 20.

3. 2.

4. `age >= 18` and `age <= 60`.

5. `==` compares; `=` assigns.

6. `x ** 2`.

7. `total += 5`.

8. False.

Independent Practice

- A. Write three short code examples using Operators and Expressions.
- B. Change one example so it handles a different input or condition.
- C. Write one deliberately incorrect example, then correct it and explain the error.
- D. Use the concept in a short real-life programming task.

Exam Strategy

- Read the complete problem before choosing a structure.
- Trace small examples by hand when output is uncertain.
- When two solutions work, explain differences in clarity, efficiency, or robustness.

Notes & Corrections

Record one mistake you made, the correction, and the programming rule or idea.

LESSON 06 - CORE PROGRAMMING

Input, Output and Type Conversion

Concepts, syntax, and model examples

Learning Objectives

- Read user input and display formatted output.
- Convert between text and numeric types.
- Validate simple input assumptions.

Concept	Explanation	Example
Input	Reads data supplied to a program.	<code>name = input("Name: ")</code>
Output	Displays information to the user.	<code>print("Hello", name)</code>
Conversion	Changes data from one type to another.	<code>age = int(input("Age: "))</code>
Formatting	Builds readable output with values.	<code>print(f"Total: {total:.2f}")</code>

Key Rules

- Python `input()` returns text, so convert before arithmetic when needed.
- Use clear prompts that tell the user what to enter.
- Format decimals to a sensible number of places for presentation.

Study Tip

Type each model example yourself. Change one value or condition and predict the result before running it.

Extra Model Code / Sentences

```
1. name = input("Enter your name: ")
2. x = float(input("Enter x: "))
3. y = float(input("Enter y: "))
4. print(f"Sum = {x+y:.2f}")
```

Worked Analysis Routine

- Step 1: Identify the inputs, outputs, and target operation.
- Step 2: Trace variables and conditions in execution order.
- Step 3: Explain why the chosen syntax or structure matches the intended logic.
- Step 4: Modify one input or rule and predict how the output changes.

Your Example

Write one accurate example or short code fragment and explain the target programming idea.

LESSON 06 - PRACTICE

Input, Output and Type Conversion

Common errors, guided practice, and mastery check

Common error	Better form	Why?
<code>age = input("Age: "); next_age = age + 1</code>	<code>age = int(input("Age: ")); next_age = age + 1</code>	Convert text input before arithmetic.
<code>print("Total" total)</code>	<code>print("Total", total)</code>	Separate output items correctly.

Guided Practice

- Write input for a student name.
- Read an integer quantity.
- Read a decimal price.
- Output name and age in one line.
- Why does input() need conversion for arithmetic?
- Convert "25" to an integer.
- Convert 8 to text.
- Format 12.3456 to two decimal places.

Mastery Challenge

- Write two original examples demonstrating the main concept of this lesson.
- Trace or explain the expected output/result in one short paragraph.
- Check syntax, indentation, names, and edge cases before comparing with model answers.

Quick Answers

- `name=input("Name: ")`
- `quantity=int(input("Quantity: "))`
- `price=float(input("Price: "))`
- `print(name, age)` or f-string.
- Because input returns a string.
- `int("25") -> 25`.
- `str(8) -> "8"`.
- `f"{12.3456:.2f}" -> 12.35`.

Independent Practice

- Write three short code examples using Input, Output and Type Conversion.
- Change one example so it handles a different input or condition.
- Write one deliberately incorrect example, then correct it and explain the error.
- Use the concept in a short real-life programming task.

Exam Strategy

- Read the complete problem before choosing a structure.
- Trace small examples by hand when output is uncertain.
- When two solutions work, explain differences in clarity, efficiency, or robustness.

Notes & Corrections

Record one mistake you made, the correction, and the programming rule or idea.

LESSON 07 - CORE PROGRAMMING

Conditional Statements

Concepts, syntax, and model examples

Learning Objectives

- Use if, elif, and else for decision making.
- Write compound conditions with logical operators.
- Design complete and mutually sensible branches.

Concept	Explanation	Example
if	Runs a block when a condition is true.	if score >= 50: print("Pass")
elif	Tests another condition if earlier tests were false.	elif score >= 40: print("Resit")
else	Runs when no earlier condition matched.	else: print("Fail")
Nested selection	Places one decision inside another.	if logged_in: if admin: ...

Key Rules

- Order overlapping ranges carefully, usually from most specific/highest to lowest.
- Indent each controlled block consistently.
- Use else only when a final catch-all case is appropriate.

Study Tip

Type each model example yourself. Change one value or condition and predict the result before running it.

Extra Model Code / Sentences

```
1. if mark >= 80: grade = "A"
2. elif mark >= 60: grade = "B"
3. elif mark >= 50: grade = "C"
4. else: grade = "F"
```

Worked Analysis Routine

Step 1: Identify the inputs, outputs, and target operation.
 Step 2: Trace variables and conditions in execution order.
 Step 3: Explain why the chosen syntax or structure matches the intended logic.
 Step 4: Modify one input or rule and predict how the output changes.

Your Example

Write one accurate example or short code fragment and explain the target programming idea.

LESSON 07 - PRACTICE

Conditional Statements

Common errors, guided practice, and mastery check

Common error	Better form	Why?
if score >= 50 then	if score >= 50:	Python uses a colon, not then.
if mark >= 50: ... if mark >= 80: ...	Use elif for mutually exclusive grade bands.	Separate if statements may both run.

Guided Practice

- Write an if statement for adult age.
- Add an else branch for minor.
- Write a three-band temperature classifier.
- Use and to test a range.
- Correct: if x > 0 print("Positive").
- Explain why elif is useful.
- Write a condition for n not equal to zero.
- Give one example of nested selection.

Mastery Challenge

- Write two original examples demonstrating the main concept of this lesson.
- Trace or explain the expected output/result in one short paragraph.
- Check syntax, indentation, names, and edge cases before comparing with model answers.

Quick Answers

- if age >= 18: ...
- else: ...
- Answers vary.
- Example: 10 <= temp <= 30.
- if x > 0: print("Positive").
- It tests alternative branches without evaluating all as separate decisions.
- n != 0.
- Answers vary.

Independent Practice

- A. Write three short code examples using Conditional Statements.
- B. Change one example so it handles a different input or condition.
- C. Write one deliberately incorrect example, then correct it and explain the error.
- D. Use the concept in a short real-life programming task.

Exam Strategy

- Read the complete problem before choosing a structure.
- Trace small examples by hand when output is uncertain.
- When two solutions work, explain differences in clarity, efficiency, or robustness.

Notes & Corrections

Record one mistake you made, the correction, and the programming rule or idea.

LESSON 08 - CORE PROGRAMMING

While Loops

Concepts, syntax, and model examples

Learning Objectives

- Use while loops for condition-controlled repetition.
- Update loop variables to avoid infinite loops.
- Apply sentinel and validation loops.

Concept	Explanation	Example
while	Repeats while a condition remains true.	while count < 5: ...
Counter	Tracks the number of repetitions.	count += 1
Sentinel	A special value ends repetition.	while value != -1:
Validation loop	Repeats until input meets a rule.	while age < 0: ask again

Key Rules

- Ensure the loop condition can eventually become false.
- Initialize control variables before the loop.
- Use while when the number of repetitions is not known in advance.

Study Tip

Type each model example yourself. Change one value or condition and predict the result before running it.

Extra Model Code / Sentences

```

1. count = 1
2. while count <= 5:
3.     print(count)
4.     count += 1

```

Worked Analysis Routine

Step 1: Identify the inputs, outputs, and target operation.
 Step 2: Trace variables and conditions in execution order.
 Step 3: Explain why the chosen syntax or structure matches the intended logic.
 Step 4: Modify one input or rule and predict how the output changes.

Your Example

Write one accurate example or short code fragment and explain the target programming idea.

LESSON 08 - PRACTICE

While Loops

Common errors, guided practice, and mastery check

Common error	Better form	Why?
while count <= 5: print(count) without updating count	Add count += 1.	Otherwise the loop may never end.
Use while for a fixed simple range of 1..10	Prefer for with range when iteration count is known.	Choose the clearer loop type.

Guided Practice

- Write a while loop printing 1 to 3.
- What causes an infinite loop?
- Initialize a counter at 0 and increment it.
- Write a sentinel loop ending on 0.
- Write validation for a positive number.
- When is while preferred to for?
- Trace count values for three iterations.
- Correct a loop whose condition never changes.

Mastery Challenge

- Write two original examples demonstrating the main concept of this lesson.
- Trace or explain the expected output/result in one short paragraph.
- Check syntax, indentation, names, and edge cases before comparing with model answers.

Quick Answers

- count=1; while count<=3: print(count); count+=1.
- A condition that never becomes false.
- count=0; count+=1.
- Answers vary.
- while n <= 0: ask again.
- When repetitions depend on a condition/event.
- Depends on initialization; example 1,2,3.
- Update the controlling value or revise condition.

Independent Practice

- Write three short code examples using While Loops.
- Change one example so it handles a different input or condition.
- Write one deliberately incorrect example, then correct it and explain the error.
- Use the concept in a short real-life programming task.

Exam Strategy

- Read the complete problem before choosing a structure.
- Trace small examples by hand when output is uncertain.
- When two solutions work, explain differences in clarity, efficiency, or robustness.

Notes & Corrections

Record one mistake you made, the correction, and the programming rule or idea.

For Loops and Range

Concepts, syntax, and model examples

Learning Objectives		
- Use for loops for count-controlled repetition. - Apply range(start, stop, step) correctly. - Iterate over sequences.		
Concept	Explanation	Example
for	Repeats once for each item in a sequence.	for name in names:
range	Generates integer sequences for loops.	range(1, 6) gives 1..5
step	Controls increment or decrement.	range(10, 0, -2)
Accumulator	Combines values across iterations.	total += value

Key Rules
- The stop value in range is excluded. - Use a negative step when counting down. - Initialize accumulators such as total before the loop.
Study Tip
Type each model example yourself. Change one value or condition and predict the result before running it.

Extra Model Code / Sentences
<pre>1. total = 0 2. for n in range(1, 6): 3. total += n 4. print(total) # 15</pre>

Worked Analysis Routine
Step 1: Identify the inputs, outputs, and target operation. Step 2: Trace variables and conditions in execution order. Step 3: Explain why the chosen syntax or structure matches the intended logic. Step 4: Modify one input or rule and predict how the output changes.

Your Example
Write one accurate example or short code fragment and explain the target programming idea.

LESSON 09 - PRACTICE

For Loops and Range

Common errors, guided practice, and mastery check

Common error	Better form	Why?
range(1,5) expecting 1..5	Use range(1,6).	The stop value is excluded.
total += n before total exists	Set total = 0 first.	Accumulators need initialization.

Guided Practice

1. What numbers does range(3) produce?
2. Write a loop from 1 to 10.
3. Write a loop for even numbers 2 to 10.
4. Write a countdown 5 to 1.
5. Sum numbers 1 to 100.
6. Iterate over letters in "CODE".
7. Explain the stop parameter.
8. Choose for or while for reading exactly 20 marks.

Mastery Challenge

- Write two original examples demonstrating the main concept of this lesson.
- Trace or explain the expected output/result in one short paragraph.
- Check syntax, indentation, names, and edge cases before comparing with model answers.

Quick Answers

1. 0,1,2.
2. for i in range(1,11): ...
3. range(2,11,2).
4. range(5,0,-1).
5. Use accumulator in range(1,101).
6. for ch in "CODE": ...
7. It is excluded.
8. for.

Independent Practice

- A. Write three short code examples using For Loops and Range.
- B. Change one example so it handles a different input or condition.
- C. Write one deliberately incorrect example, then correct it and explain the error.
- D. Use the concept in a short real-life programming task.

Exam Strategy

- Read the complete problem before choosing a structure.
- Trace small examples by hand when output is uncertain.
- When two solutions work, explain differences in clarity, efficiency, or robustness.

Notes & Corrections

Record one mistake you made, the correction, and the programming rule or idea.

LESSON 10 - CORE PROGRAMMING

Strings and Text Processing

Concepts, syntax, and model examples

Learning Objectives

- Index and slice strings.
- Use common string methods.
- Build and analyse text values.

Concept	Explanation	Example
Indexing	Accesses one character by position.	word[0]
Slicing	Extracts a substring.	word[1:4]
Methods	Perform common text operations.	lower(), upper(), strip(), replace()
Membership	Tests whether text occurs inside text.	"cat" in sentence

Key Rules

- String indexes begin at 0 in Python.
- Strings are immutable; methods usually return new strings.
- Use strip() to remove surrounding whitespace from user input.

Study Tip

Type each model example yourself. Change one value or condition and predict the result before running it.

Extra Model Code / Sentences

```

1. name = "Programming"
2. print(name[0])      # P
3. print(name[0:7])    # Program
4. print(name.lower())

```

Worked Analysis Routine

Step 1: Identify the inputs, outputs, and target operation.
 Step 2: Trace variables and conditions in execution order.
 Step 3: Explain why the chosen syntax or structure matches the intended logic.
 Step 4: Modify one input or rule and predict how the output changes.

Your Example

Write one accurate example or short code fragment and explain the target programming idea.

LESSON 10 - PRACTICE

Strings and Text Processing

Common errors, guided practice, and mastery check

Common error	Better form	Why?
word[1] expecting first character	Use word[0].	Python indexing starts at 0.
word.upper without parentheses	word.upper()	Methods must be called.

Guided Practice

1. What is "CODE"[0]?
2. What is "CODE"[-1]?
3. Slice "Python" to get "Pyt".
4. Convert "Hello" to lowercase.
5. Remove spaces around " test ".
6. Test whether "a" is in "data".
7. Count the length of a string.
8. Explain string immutability.

Mastery Challenge

- Write two original examples demonstrating the main concept of this lesson.
- Trace or explain the expected output/result in one short paragraph.
- Check syntax, indentation, names, and edge cases before comparing with model answers.

Quick Answers

1. C.
2. E.
3. "Python"[0:3].
4. "Hello".lower().
5. " test ".strip().
6. "a" in "data" -> True.
7. len(text).
8. Existing string characters cannot be changed in place.

Independent Practice

- A. Write three short code examples using Strings and Text Processing.
- B. Change one example so it handles a different input or condition.
- C. Write one deliberately incorrect example, then correct it and explain the error.
- D. Use the concept in a short real-life programming task.

Exam Strategy

- Read the complete problem before choosing a structure.
- Trace small examples by hand when output is uncertain.
- When two solutions work, explain differences in clarity, efficiency, or robustness.

Notes & Corrections

Record one mistake you made, the correction, and the programming rule or idea.

LESSON 11 - CORE PROGRAMMING

Lists and Arrays

Concepts, syntax, and model examples

Learning Objectives		
• Create, index, modify, and traverse lists. • Use common list methods. • Apply lists to collections of related values.		
Concept	Explanation	Example
List	Ordered mutable collection.	marks = [12, 15, 9]
Indexing	Accesses or changes an item.	marks[1] = 16
Methods	Add, remove, sort, or inspect.	append(), remove(), sort()
Traversal	Processes each element in turn.	for mark in marks: ...

Key Rules

- List indexes start at 0.
- append adds one item to the end.
- Check that an index exists before accessing it in uncertain situations.

Study Tip

Type each model example yourself. Change one value or condition and predict the result before running it.

Extra Model Code / Sentences

```
1. marks = [10, 15, 12]
2. marks.append(18)
3. marks[0] = 11
4. average = sum(marks) / len(marks)
```

Worked Analysis Routine

Step 1: Identify the inputs, outputs, and target operation.

Step 2: Trace variables and conditions in execution order.

Step 3: Explain why the chosen syntax or structure matches the intended logic.

Step 4: Modify one input or rule and predict how the output changes.

Your Example

Write one accurate example or short code fragment and explain the target programming idea.

LESSON 11 - PRACTICE

Lists and Arrays

Common errors, guided practice, and mastery check

Common error	Better form	Why?
marks[3] on a three-item list	Use indexes 0,1,2.	Index 3 is out of range.
marks = marks.append(10)	marks.append(10)	append modifies the list and returns None.

Guided Practice

1. Create a list of three names.
2. Access the second item.
3. Change the first item.
4. Append one item.
5. Remove a value.
6. Sort a numeric list.
7. Calculate length.
8. Loop over all items.

Mastery Challenge

- Write two original examples demonstrating the main concept of this lesson.
- Trace or explain the expected output/result in one short paragraph.
- Check syntax, indentation, names, and edge cases before comparing with model answers.

Quick Answers

1. Example: ["Ana","Bo","Cy"].
2. items[1].
3. items[0]=...
4. items.append(x).
5. items.remove(x).
6. items.sort().
7. len(items).
8. for item in items: ...

Independent Practice

- A. Write three short code examples using Lists and Arrays.
- B. Change one example so it handles a different input or condition.
- C. Write one deliberately incorrect example, then correct it and explain the error.
- D. Use the concept in a short real-life programming task.

Exam Strategy

- Read the complete problem before choosing a structure.
- Trace small examples by hand when output is uncertain.
- When two solutions work, explain differences in clarity, efficiency, or robustness.

Notes & Corrections

Record one mistake you made, the correction, and the programming rule or idea.

Tuples, Sets and Dictionaries

Concepts, syntax, and model examples

Learning Objectives

- Distinguish common Python collection types.
- Use dictionaries for key-value data.
- Use sets for unique membership.

Concept	Explanation	Example
Tuple	Ordered immutable collection.	point = (3, 4)
Set	Unordered collection of unique values.	colors = {"red", "blue"}
Dictionary	Stores key-value pairs.	student = {"name":"Mia", "age":17}
Lookup	Accesses data by key or membership.	student["name"]; "red" in colors

Key Rules

- Choose tuples for fixed records, sets for uniqueness, dictionaries for named lookup.
- Dictionary keys must be hashable and unique.
- Sets automatically remove duplicate values.

Study Tip

Type each model example yourself. Change one value or condition and predict the result before running it.

Extra Model Code / Sentences

```
1. student = {"name":"Lina", "score":88}
2. student["score"] = 91
3. subjects = {"Math", "Physics", "Math"}
4. print(subjects) # duplicates removed
```

Worked Analysis Routine

Step 1: Identify the inputs, outputs, and target operation.
Step 2: Trace variables and conditions in execution order.
Step 3: Explain why the chosen syntax or structure matches the intended logic.
Step 4: Modify one input or rule and predict how the output changes.

Your Example

Write one accurate example or short code fragment and explain the target programming idea.

LESSON 12 - PRACTICE

Tuples, Sets and Dictionaries

Common errors, guided practice, and mastery check

Common error	Better form	Why?
student[0] expecting first dictionary field	Use a key such as student["name"].	Dictionaries are accessed by keys.
Try to assign tuple[0] = 5	Create a new tuple or use a list.	Tuples are immutable.

Guided Practice

1. Create a two-value tuple.
2. Create a set with repeated numbers.
3. Create a dictionary for name and age.
4. Read a value by key.
5. Add a new dictionary key.
6. What happens to duplicate set values?
7. Which collection is immutable?
8. Choose the best collection for unique usernames.

Mastery Challenge

- Write two original examples demonstrating the main concept of this lesson.
- Trace or explain the expected output/result in one short paragraph.
- Check syntax, indentation, names, and edge cases before comparing with model answers.

Quick Answers

1. Example: (2,3).
2. {1,2,2} becomes {1,2}.
3. {"name": "...", "age": ...}.
4. d["key"].
5. d["city"]="...".
6. Duplicates are removed.
7. Tuple.
8. Set.

Independent Practice

- A. Write three short code examples using Tuples, Sets and Dictionaries.
- B. Change one example so it handles a different input or condition.
- C. Write one deliberately incorrect example, then correct it and explain the error.
- D. Use the concept in a short real-life programming task.

Exam Strategy

- Read the complete problem before choosing a structure.
- Trace small examples by hand when output is uncertain.
- When two solutions work, explain differences in clarity, efficiency, or robustness.

Notes & Corrections

Record one mistake you made, the correction, and the programming rule or idea.

LESSON 13 - CORE PROGRAMMING

Functions

Concepts, syntax, and model examples

Learning Objectives

- Define and call reusable functions.
- Use parameters and return values.
- Decompose a program into smaller tasks.

Concept	Explanation	Example
Definition	Creates a named block of reusable code.	<code>def greet(): ...</code>
Parameter	Receives input supplied by the caller.	<code>def square(x):</code>
Return value	Sends a result back to the caller.	<code>return x * x</code>
Decomposition	Splits a large problem into smaller functions.	<code>input_data(), calculate(), display()</code>

Key Rules

- A function should have one clear purpose.
- Use return when a calculated value is needed elsewhere.
- Function names should describe actions, such as `calculate_total`.

Study Tip

Type each model example yourself. Change one value or condition and predict the result before running it.

Extra Model Code / Sentences

```

1. def area(length, width):
2.     return length * width
3. result = area(5, 3)
4. print(result)

```

Worked Analysis Routine

Step 1: Identify the inputs, outputs, and target operation.
 Step 2: Trace variables and conditions in execution order.
 Step 3: Explain why the chosen syntax or structure matches the intended logic.
 Step 4: Modify one input or rule and predict how the output changes.

Your Example

Write one accurate example or short code fragment and explain the target programming idea.

LESSON 13 - PRACTICE

Functions

Common errors, guided practice, and mastery check

Common error	Better form	Why?
Define function but never call it	Call the function where needed.	A definition alone does not execute its body.
print result inside every calculation function	Return result when caller needs it.	Returning improves reuse.

Guided Practice

1. Define a function `hello()`.
2. Define `square(x)`.
3. Call `square` with 5.
4. Explain parameter versus argument.
5. Write a function that returns the larger of two numbers.
6. Why use functions?
7. What happens after return?
8. Give one good function name for converting Celsius to Fahrenheit.

Mastery Challenge

- Write two original examples demonstrating the main concept of this lesson.
- Trace or explain the expected output/result in one short paragraph.
- Check syntax, indentation, names, and edge cases before comparing with model answers.

Quick Answers

1. `def hello(): ...`
2. `def square(x): return x*x.`
3. `square(5).`
4. Parameter is in definition; argument is supplied in call.
5. Answers vary.
6. Reuse, clarity, testing, decomposition.
7. The function ends and passes back a value.
8. `celsius_to_fahrenheit`.

Independent Practice

- A. Write three short code examples using Functions.
- B. Change one example so it handles a different input or condition.
- C. Write one deliberately incorrect example, then correct it and explain the error.
- D. Use the concept in a short real-life programming task.

Exam Strategy

- Read the complete problem before choosing a structure.
- Trace small examples by hand when output is uncertain.
- When two solutions work, explain differences in clarity, efficiency, or robustness.

Notes & Corrections

Record one mistake you made, the correction, and the programming rule or idea.

LESSON 14 - CORE PROGRAMMING

Parameters, Return Values and Scope

Concepts, syntax, and model examples

Learning Objectives

- Use positional and keyword arguments.
- Understand local and global scope.
- Return multiple or structured results appropriately.

Concept	Explanation	Example
Positional argument	Matched by position.	<code>power(2, 3)</code>
Keyword argument	Matched by parameter name.	<code>power(base=2, exponent=3)</code>
Local variable	Exists inside a function.	<code>total</code> inside <code>calculate()</code>
Global variable	Defined at module level; use cautiously.	<code>PI = 3.14159</code>

Key Rules

- Prefer local variables to reduce unintended side effects.
- Return data instead of modifying globals when practical.
- Default parameters should represent sensible common behavior.

Study Tip

Type each model example yourself. Change one value or condition and predict the result before running it.

Extra Model Code / Sentences

```

1. def greet(name, prefix="Hello"):
2.     message = f"{prefix}, {name}"
3.     return message
4. print(greet("Tiana"))

```

Worked Analysis Routine

Step 1: Identify the inputs, outputs, and target operation.
 Step 2: Trace variables and conditions in execution order.
 Step 3: Explain why the chosen syntax or structure matches the intended logic.
 Step 4: Modify one input or rule and predict how the output changes.

Your Example

Write one accurate example or short code fragment and explain the target programming idea.

LESSON 14 - PRACTICE

Parameters, Return Values and Scope

Common errors, guided practice, and mastery check

Common error	Better form	Why?
Use a local variable outside its function	Return it or define it in the needed scope.	Local scope ends after function execution.
Use many globals for communication	Pass parameters and return values.	This makes dependencies clearer.

Guided Practice

1. What is local scope?
2. Call greet using a keyword argument.
3. Define add(a,b=0).
4. Explain why globals can be risky.
5. Return two values from a function conceptually.
6. What is a default parameter?
7. Which variable is visible only inside a function?
8. Rewrite a global dependency using a parameter.

Mastery Challenge

- Write two original examples demonstrating the main concept of this lesson.
- Trace or explain the expected output/result in one short paragraph.
- Check syntax, indentation, names, and edge cases before comparing with model answers.

Quick Answers

1. Variables defined inside a function are local by default.
2. greet(name="Tiana").
3. def add(a,b=0): return a+b.
4. They create hidden dependencies/side effects.
5. return x, y.
6. A parameter with a value used when caller omits it.
7. Local variable.
8. Answers vary.

Independent Practice

- A. Write three short code examples using Parameters, Return Values and Scope.
- B. Change one example so it handles a different input or condition.
- C. Write one deliberately incorrect example, then correct it and explain the error.
- D. Use the concept in a short real-life programming task.

Exam Strategy

- Read the complete problem before choosing a structure.
- Trace small examples by hand when output is uncertain.
- When two solutions work, explain differences in clarity, efficiency, or robustness.

Notes & Corrections

Record one mistake you made, the correction, and the programming rule or idea.

LESSON 15 - CORE PROGRAMMING

Recursion

Concepts, syntax, and model examples

Learning Objectives

- Explain recursive problem solving.
- Identify base and recursive cases.
- Trace simple recursive calls safely.

Concept	Explanation	Example
Recursive function	Calls itself on a smaller version of a problem.	factorial(n) uses factorial(n-1)
Base case	Stops further recursive calls.	if n <= 1: return 1
Recursive case	Reduces the problem.	return n * factorial(n-1)
Call stack	Stores active function calls.	factorial(4) waits for factorial(3), etc.

Key Rules

- Every recursion must reach a base case.
- Reduce the problem size on each recursive call.
- Use iteration when it is simpler and more memory-efficient.

Study Tip

Type each model example yourself. Change one value or condition and predict the result before running it.

Extra Model Code / Sentences

```

1. def factorial(n):
2.     if n <= 1: return 1
3.     return n * factorial(n-1)
4. factorial(4) -> 24

```

Worked Analysis Routine

Step 1: Identify the inputs, outputs, and target operation.
 Step 2: Trace variables and conditions in execution order.
 Step 3: Explain why the chosen syntax or structure matches the intended logic.
 Step 4: Modify one input or rule and predict how the output changes.

Your Example

Write one accurate example or short code fragment and explain the target programming idea.

LESSON 15 - PRACTICE

Recursion

Common errors, guided practice, and mastery check

Common error	Better form	Why?
Recursive call does not move toward base case	Reduce n or other problem measure.	Otherwise recursion may never terminate.
No base case	Add a stopping condition.	Base case prevents infinite recursion.

Guided Practice

1. Define recursion.
2. What is a base case?
3. Trace factorial(3).
4. Why can recursion cause a stack overflow?
5. Write recursive sum 1..n conceptually.
6. State one recursive problem example.
7. When may iteration be better?
8. Identify the recursive call in factorial code.

Mastery Challenge

- Write two original examples demonstrating the main concept of this lesson.
- Trace or explain the expected output/result in one short paragraph.
- Check syntax, indentation, names, and edge cases before comparing with model answers.

Quick Answers

1. A function solving a problem by calling itself on smaller input.
2. Stopping case with no further self-call.
3. $3 \times 2 \times 1 = 6$.
4. Too many nested calls use stack memory.
5. $\text{sum}(n) = n + \text{sum}(n-1)$, base $n \leq 0$.
6. Tree traversal, factorial, Fibonacci, etc.
7. When a simple loop is clearer or more efficient.
8. $\text{factorial}(n-1)$.

Independent Practice

- A. Write three short code examples using Recursion.
- B. Change one example so it handles a different input or condition.
- C. Write one deliberately incorrect example, then correct it and explain the error.
- D. Use the concept in a short real-life programming task.

Exam Strategy

- Read the complete problem before choosing a structure.
- Trace small examples by hand when output is uncertain.
- When two solutions work, explain differences in clarity, efficiency, or robustness.

Notes & Corrections

Record one mistake you made, the correction, and the programming rule or idea.

Modules and Libraries

Concepts, syntax, and model examples

Learning Objectives		
• Import and use standard library modules. • Organize reusable code in modules. • Read module documentation and call functions correctly.		
Concept	Explanation	Example
Module	A Python file containing reusable code.	math, random, statistics
Import	Makes module names available.	import math
Selective import	Imports specific members.	from math import sqrt
Namespace	Prevents name collisions by qualifying names.	math.sqrt(25)

Key Rules

- Prefer explicit module names for clarity in larger programs.
- Avoid from module import * because it hides origins and may cause collisions.
- Use official documentation to check function behavior and parameters.

Study Tip

Type each model example yourself. Change one value or condition and predict the result before running it.

Extra Model Code / Sentences

```
1. import math
2. radius = 3
3. area = math.pi * radius ** 2
4. print(math.sqrt(81))
```

Worked Analysis Routine

Step 1: Identify the inputs, outputs, and target operation.

Step 2: Trace variables and conditions in execution order.

Step 3: Explain why the chosen syntax or structure matches the intended logic.

Step 4: Modify one input or rule and predict how the output changes.

Your Example

Write one accurate example or short code fragment and explain the target programming idea.

LESSON 16 - PRACTICE

Modules and Libraries

Common errors, guided practice, and mastery check

Common error	Better form	Why?
Call <code>sqrt</code> without importing it	Use <code>math.sqrt</code> after <code>import math</code> .	The name must exist in the namespace.
Use <code>import *</code> in complex code	Import module or selected names explicitly.	Explicit imports improve readability.

Guided Practice

1. Import `math`.
2. Use `math.pi`.
3. Generate a random integer conceptually.
4. What is a module?
5. Explain namespace.
6. Why avoid `import *`?
7. Name two standard library modules.
8. State one benefit of organizing your own functions into modules.

Mastery Challenge

- Write two original examples demonstrating the main concept of this lesson.
- Trace or explain the expected output/result in one short paragraph.
- Check syntax, indentation, names, and edge cases before comparing with model answers.

Quick Answers

1. `import math`.
2. `math.pi`.
3. `random.randint(a,b)` after `import random`.
4. Reusable code packaged in a file/module.
5. A mapping of names that helps avoid collisions.
6. It can hide where names came from and cause collisions.
7. `math`, `random`, `statistics`, `datetime`, etc.
8. Reuse and organization.

Independent Practice

- A. Write three short code examples using Modules and Libraries.
- B. Change one example so it handles a different input or condition.
- C. Write one deliberately incorrect example, then correct it and explain the error.
- D. Use the concept in a short real-life programming task.

Exam Strategy

- Read the complete problem before choosing a structure.
- Trace small examples by hand when output is uncertain.
- When two solutions work, explain differences in clarity, efficiency, or robustness.

Notes & Corrections

Record one mistake you made, the correction, and the programming rule or idea.

File Handling

Concepts, syntax, and model examples

Learning Objectives

- Open, read, write, and close text files safely.
- Use with blocks for automatic resource management.
- Process file data line by line.

Concept	Explanation	Example
Read mode	Opens a file for reading.	open("data.txt", "r")
Write mode	Creates/replaces a file for writing.	open("out.txt", "w")
Append mode	Adds data at the end.	open("log.txt", "a")
with block	Closes the file automatically.	with open(...) as f: ...

Key Rules

- Use with open(...) as f for safe file handling.
- Writing with mode w can overwrite existing content.
- Strip newline characters when comparing line text when needed.

Study Tip

Type each model example yourself. Change one value or condition and predict the result before running it.

Extra Model Code / Sentences

```
1. with open("marks.txt", "r") as f:
2.     for line in f:
3.         mark = float(line.strip())
4.         print(mark)
```

Worked Analysis Routine

- Step 1: Identify the inputs, outputs, and target operation.
- Step 2: Trace variables and conditions in execution order.
- Step 3: Explain why the chosen syntax or structure matches the intended logic.
- Step 4: Modify one input or rule and predict how the output changes.

Your Example

Write one accurate example or short code fragment and explain the target programming idea.

LESSON 17 - PRACTICE

File Handling

Common errors, guided practice, and mastery check

Common error	Better form	Why?
Open a file and forget to close it	Use a with block.	The context manager closes it automatically.
Use w when intending to preserve existing data	Use a for append.	w replaces existing content.

Guided Practice

1. Open notes.txt for reading.
2. Open report.txt for writing.
3. Append to log.txt.
4. Explain with open.
5. Read all lines conceptually.
6. Write one line to a file.
7. Why can w be dangerous?
8. Remove newline spaces from a read line.

Mastery Challenge

- Write two original examples demonstrating the main concept of this lesson.
- Trace or explain the expected output/result in one short paragraph.
- Check syntax, indentation, names, and edge cases before comparing with model answers.

Quick Answers

1. with open("notes.txt","r") as f: ...
2. with open("report.txt","w") as f: ...
3. open(...,"a").
4. It manages the resource and closes automatically.
5. f.readlines() or loop.
6. f.write("text\n").
7. It can overwrite the file.
8. line.strip().

Independent Practice

- A. Write three short code examples using File Handling.
- B. Change one example so it handles a different input or condition.
- C. Write one deliberately incorrect example, then correct it and explain the error.
- D. Use the concept in a short real-life programming task.

Exam Strategy

- Read the complete problem before choosing a structure.
- Trace small examples by hand when output is uncertain.
- When two solutions work, explain differences in clarity, efficiency, or robustness.

Notes & Corrections

Record one mistake you made, the correction, and the programming rule or idea.

LESSON 18 - CORE PROGRAMMING

Exceptions and Debugging

Concepts, syntax, and model examples

Learning Objectives		
- Identify syntax, runtime, and logic errors. - Handle predictable runtime errors using try/except. - Use a systematic debugging process.		
Concept	Explanation	Example
Syntax error	Breaks grammar rules of the language.	Missing colon or unmatched parenthesis
Runtime error	Occurs while program runs.	Division by zero, invalid conversion
Logic error	Program runs but result is wrong.	Using + instead of *
Exception handling	Catches selected runtime exceptions.	try: ... except ValueError: ...

Key Rules
- Read the error message and traceback before changing code. - Catch specific exceptions instead of using a broad except when possible. - Use small tests and print/log values to isolate the failing step.
Study Tip
Type each model example yourself. Change one value or condition and predict the result before running it.

Extra Model Code / Sentences
<pre>1. try: 2. age = int(input("Age: ")) 3. except ValueError: 4. print("Please enter a whole number.")</pre>

Worked Analysis Routine
Step 1: Identify the inputs, outputs, and target operation. Step 2: Trace variables and conditions in execution order. Step 3: Explain why the chosen syntax or structure matches the intended logic. Step 4: Modify one input or rule and predict how the output changes.

Your Example
Write one accurate example or short code fragment and explain the target programming idea.

LESSON 18 - PRACTICE

Exceptions and Debugging

Common errors, guided practice, and mastery check

Common error	Better form	Why?
except: pass	Catch specific exceptions and handle them meaningfully.	Silent failure hides problems.
Change many lines before retesting	Make one focused change at a time.	This helps identify the actual cause.

Guided Practice

1. Classify a missing colon error.
2. Classify division by zero.
3. Classify wrong formula.
4. What does try/except do?
5. Catch ValueError for int conversion.
6. Why inspect a traceback?
7. State one debugging technique.
8. Why are specific exceptions preferred?

Mastery Challenge

- Write two original examples demonstrating the main concept of this lesson.
- Trace or explain the expected output/result in one short paragraph.
- Check syntax, indentation, names, and edge cases before comparing with model answers.

Quick Answers

1. Syntax error.
2. Runtime error/ZeroDivisionError.
3. Logic error.
4. Handles selected runtime exceptions.
5. try/except ValueError.
6. It shows where and how the failure occurred.
7. Trace variables, break problem down, use tests, debugger, etc.
8. They avoid hiding unrelated problems.

Independent Practice

- A. Write three short code examples using Exceptions and Debugging.
- B. Change one example so it handles a different input or condition.
- C. Write one deliberately incorrect example, then correct it and explain the error.
- D. Use the concept in a short real-life programming task.

Exam Strategy

- Read the complete problem before choosing a structure.
- Trace small examples by hand when output is uncertain.
- When two solutions work, explain differences in clarity, efficiency, or robustness.

Notes & Corrections

Record one mistake you made, the correction, and the programming rule or idea.

LESSON 19 - CORE PROGRAMMING

Testing and Test Cases

Concepts, syntax, and model examples

Learning Objectives

- Design normal, boundary, and invalid test cases.
- State expected results before running tests.
- Use assertions for simple automated checks.

Concept	Explanation	Example
Normal case	Typical valid input.	age=25
Boundary case	Input at or near a limit.	age=17,18 for adult test
Invalid case	Input that violates assumptions.	age="abc"
Assertion	Checks a condition during testing.	assert square(4) == 16

Key Rules

- Every test should have an expected result.
- Test boundaries where comparison operators change behavior.
- Keep regression tests so fixed bugs do not reappear.

Study Tip

Type each model example yourself. Change one value or condition and predict the result before running it.

Extra Model Code / Sentences

```
1. def is_adult(age): return age >= 18
2. assert is_adult(18) is True
3. assert is_adult(17) is False
4. assert is_adult(30) is True
```

Worked Analysis Routine

Step 1: Identify the inputs, outputs, and target operation.
 Step 2: Trace variables and conditions in execution order.
 Step 3: Explain why the chosen syntax or structure matches the intended logic.
 Step 4: Modify one input or rule and predict how the output changes.

Your Example

Write one accurate example or short code fragment and explain the target programming idea.

LESSON 19 - PRACTICE

Testing and Test Cases

Common errors, guided practice, and mastery check

Common error	Better form	Why?
Test only one normal value	Add boundary and invalid cases.	Many bugs occur at limits.
Run tests without expected outputs	Write expected result first.	Otherwise success is hard to judge.

Guided Practice

1. Give a normal test for 0-100 score.
2. Give two boundary tests.
3. Give one invalid test.
4. What is an assertion?
5. Why keep regression tests?
6. Design tests for password length ≥ 8 .
7. What should every test case include?
8. Explain black-box testing simply.

Mastery Challenge

- Write two original examples demonstrating the main concept of this lesson.
- Trace or explain the expected output/result in one short paragraph.
- Check syntax, indentation, names, and edge cases before comparing with model answers.

Quick Answers

1. Example 65.
2. 0 and 100, plus near decision boundaries.
3. -1, 101, text, etc.
4. A check that a condition/result is true.
5. To ensure old bugs do not return.
6. Lengths 7,8,9 plus invalid types.
7. Input, expected result, actual result/status.
8. Testing behavior from inputs/outputs without relying on internal implementation.

Independent Practice

- A. Write three short code examples using Testing and Test Cases.
- B. Change one example so it handles a different input or condition.
- C. Write one deliberately incorrect example, then correct it and explain the error.
- D. Use the concept in a short real-life programming task.

Exam Strategy

- Read the complete problem before choosing a structure.
- Trace small examples by hand when output is uncertain.
- When two solutions work, explain differences in clarity, efficiency, or robustness.

Notes & Corrections

Record one mistake you made, the correction, and the programming rule or idea.

LESSON 20 - CORE PROGRAMMING

Object-Oriented Programming Concepts

Concepts, syntax, and model examples

Learning Objectives

- Explain objects, classes, attributes, and methods.
- Describe encapsulation and abstraction.
- Recognize when an object model is useful.

Concept	Explanation	Example
Class	Blueprint describing data and behavior.	class Student: ...
Object	Instance created from a class.	s1 = Student(...)
Attribute	Data stored in an object.	s1.name
Method	Function associated with a class/object.	s1.display()

Key Rules

- Model related data and behavior together when it improves clarity.
- Use classes for many similar entities with shared behavior.
- Do not use OOP only to make small problems more complicated.

Study Tip

Type each model example yourself. Change one value or condition and predict the result before running it.

Extra Model Code / Sentences

```
1. class Student:
2.     def __init__(self, name): self.name = name
3.     def greet(self): return f"Hi {self.name}"
4. s = Student("Mia")
```

Worked Analysis Routine

- Step 1: Identify the inputs, outputs, and target operation.
- Step 2: Trace variables and conditions in execution order.
- Step 3: Explain why the chosen syntax or structure matches the intended logic.
- Step 4: Modify one input or rule and predict how the output changes.

Your Example

Write one accurate example or short code fragment and explain the target programming idea.

LESSON 20 - PRACTICE

Object-Oriented Programming Concepts

Common errors, guided practice, and mastery check

Common error	Better form	Why?
Confuse class with object	Class is blueprint; object is an instance.	They are related but different.
Use class for a one-line calculation	Use a simple function when appropriate.	Choose the simplest suitable abstraction.

Guided Practice

1. Define a class in words.
2. Define an object.
3. What is an attribute?
4. What is a method?
5. Give one real-world class example.
6. What does self refer to?
7. Why use encapsulation?
8. When may a function be better than a class?

Mastery Challenge

- Write two original examples demonstrating the main concept of this lesson.
- Trace or explain the expected output/result in one short paragraph.
- Check syntax, indentation, names, and edge cases before comparing with model answers.

Quick Answers

1. A blueprint/type describing objects.
2. An instance of a class.
3. Object data/state.
4. Behavior/function defined for a class.
5. Student, BankAccount, Car, etc.
6. The current instance.
7. To group/control access to related data and behavior.
8. For a small stateless operation.

Independent Practice

- A. Write three short code examples using Object-Oriented Programming Concepts.
- B. Change one example so it handles a different input or condition.
- C. Write one deliberately incorrect example, then correct it and explain the error.
- D. Use the concept in a short real-life programming task.

Exam Strategy

- Read the complete problem before choosing a structure.
- Trace small examples by hand when output is uncertain.
- When two solutions work, explain differences in clarity, efficiency, or robustness.

Notes & Corrections

Record one mistake you made, the correction, and the programming rule or idea.

LESSON 21 - CORE PROGRAMMING

Classes and Objects in Python

Concepts, syntax, and model examples

Learning Objectives		
- Define classes with constructors. - Create and use multiple objects. - Update and inspect object state.		
Concept	Explanation	Example
Constructor	Initializes new objects.	def __init__(self, ...):
Instance attribute	Stores data unique to each object.	self.balance
Instance method	Operates on one object.	deposit(self, amount)
Instantiation	Creates a new object.	acc = BankAccount(100)

Key Rules
- Initialize required attributes in __init__. - Use self.attribute for instance state. - Validate state changes inside methods when appropriate.
Study Tip
Type each model example yourself. Change one value or condition and predict the result before running it.

Extra Model Code / Sentences
<pre>1. class Counter: 2. def __init__(self): self.value = 0 3. def increment(self): self.value += 1 4. c = Counter(); c.increment()</pre>

Worked Analysis Routine
Step 1: Identify the inputs, outputs, and target operation. Step 2: Trace variables and conditions in execution order. Step 3: Explain why the chosen syntax or structure matches the intended logic. Step 4: Modify one input or rule and predict how the output changes.

Your Example
Write one accurate example or short code fragment and explain the target programming idea.

LESSON 21 - PRACTICE

Classes and Objects in Python

Common errors, guided practice, and mastery check

Common error	Better form	Why?
Forget self in instance method definition	def increment(self): ...	Instance methods receive the current object.
Assign value instead of self.value	Use self.value for persistent object state.	Local variables disappear after method returns.

Guided Practice

1. Write a class Book with title.
2. Create one Book object.
3. Add a method display_title.
4. What does __init__ do?
5. Why use self.title?
6. Create two independent Counter objects.
7. What is instantiation?
8. Explain instance attribute.

Mastery Challenge

- Write two original examples demonstrating the main concept of this lesson.
- Trace or explain the expected output/result in one short paragraph.
- Check syntax, indentation, names, and edge cases before comparing with model answers.

Quick Answers

1. class Book: def __init__(self,title): self.title=title.
2. b=Book("...").
3. def display_title(self): print(self.title).
4. Initializes a new object.
5. It stores state on the object.
6. c1=Counter(); c2=Counter().
7. Creating an object from a class.
8. A value belonging to one object instance.

Independent Practice

- A. Write three short code examples using Classes and Objects in Python.
- B. Change one example so it handles a different input or condition.
- C. Write one deliberately incorrect example, then correct it and explain the error.
- D. Use the concept in a short real-life programming task.

Exam Strategy

- Read the complete problem before choosing a structure.
- Trace small examples by hand when output is uncertain.
- When two solutions work, explain differences in clarity, efficiency, or robustness.

Notes & Corrections

Record one mistake you made, the correction, and the programming rule or idea.

Inheritance and Polymorphism

Concepts, syntax, and model examples

Learning Objectives

- Create subclasses that reuse parent behavior.
- Override methods in subclasses.
- Explain polymorphism with a common interface.

Concept	Explanation	Example
Inheritance	Subclass reuses/extends a parent class.	class Dog(Animal):
Override	Subclass provides a new method implementation.	Dog.speak()
super()	Accesses parent behavior.	super().__init__(name)
Polymorphism	Different objects respond to the same method call.	animal.speak()

Key Rules

- Use inheritance only when there is a genuine "is-a" relationship.
- Prefer composition when an object merely uses another object.
- Keep parent interfaces consistent for polymorphic use.

Study Tip

Type each model example yourself. Change one value or condition and predict the result before running it.

Extra Model Code / Sentences

```
1. class Animal:
2.     def speak(self): return "..."
3. class Dog(Animal):
4.     def speak(self): return "Woof"
```

Worked Analysis Routine

- Step 1: Identify the inputs, outputs, and target operation.
- Step 2: Trace variables and conditions in execution order.
- Step 3: Explain why the chosen syntax or structure matches the intended logic.
- Step 4: Modify one input or rule and predict how the output changes.

Your Example

Write one accurate example or short code fragment and explain the target programming idea.

LESSON 22 - PRACTICE

Inheritance and Polymorphism

Common errors, guided practice, and mastery check

Common error	Better form	Why?
Use inheritance for unrelated classes	Use composition or separate classes.	Inheritance should model an is-a relationship.
Duplicate parent initialization manually	Use <code>super()</code> where appropriate.	This reduces duplication.

Guided Practice

1. What is a subclass?
2. What is overriding?
3. What does `super()` do?
4. Give an is-a example.
5. Give a has-a composition example.
6. Explain polymorphism simply.
7. Why avoid deep inheritance chains?
8. Write a subclass `Cat` of `Animal` conceptually.

Mastery Challenge

- Write two original examples demonstrating the main concept of this lesson.
- Trace or explain the expected output/result in one short paragraph.
- Check syntax, indentation, names, and edge cases before comparing with model answers.

Quick Answers

1. A class derived from another class.
2. Replacing/inserting subclass behavior for an inherited method.
3. Accesses parent class behavior.
4. Dog is an Animal.
5. Car has an Engine.
6. Same interface, different implementations.
7. They become difficult to understand and maintain.
8. `class Cat(Animal): ...`

Independent Practice

- A. Write three short code examples using Inheritance and Polymorphism.
- B. Change one example so it handles a different input or condition.
- C. Write one deliberately incorrect example, then correct it and explain the error.
- D. Use the concept in a short real-life programming task.

Exam Strategy

- Read the complete problem before choosing a structure.
- Trace small examples by hand when output is uncertain.
- When two solutions work, explain differences in clarity, efficiency, or robustness.

Notes & Corrections

Record one mistake you made, the correction, and the programming rule or idea.

LESSON 23 - CORE PROGRAMMING

Searching Algorithms

Concepts, syntax, and model examples

Learning Objectives

- Implement linear search.
- Explain binary search requirements.
- Compare linear and binary search conceptually.

Concept	Explanation	Example
Linear search	Checks items one by one.	Search from first item until match/end.
Binary search	Repeatedly halves a sorted search interval.	Compare with middle element.
Found index	Position where target occurs.	Return i when items[i] == target.
Precondition	Binary search requires sorted data.	[2,4,7,9,12]

Key Rules

- Use linear search for small or unsorted data.
- Binary search requires sorted ordered data.
- Decide whether to return index, Boolean, or object and document the choice.

Study Tip

Type each model example yourself. Change one value or condition and predict the result before running it.

Extra Model Code / Sentences

```

1. def linear_search(items, target):
2.     for i, value in enumerate(items):
3.         if value == target: return i
4.     return -1

```

Worked Analysis Routine

Step 1: Identify the inputs, outputs, and target operation.
 Step 2: Trace variables and conditions in execution order.
 Step 3: Explain why the chosen syntax or structure matches the intended logic.
 Step 4: Modify one input or rule and predict how the output changes.

Your Example

Write one accurate example or short code fragment and explain the target programming idea.

LESSON 23 - PRACTICE

Searching Algorithms

Common errors, guided practice, and mastery check

Common error	Better form	Why?
Use binary search on unsorted data	Sort first or use linear search.	Binary search relies on order.
Return 0 for not found	Use -1 or None consistently.	0 may be a valid found index.

Guided Practice

- Describe linear search.
- What is required for binary search?
- Search [4,7,9] for 7 using linear search.
- What index should be returned?
- When is linear search convenient?
- Why is binary search faster on large sorted data?
- What can represent not found?
- State one cost of sorting before binary search.

Mastery Challenge

- Write two original examples demonstrating the main concept of this lesson.
- Trace or explain the expected output/result in one short paragraph.
- Check syntax, indentation, names, and edge cases before comparing with model answers.

Quick Answers

- Check each item in order.
- Sorted data.
- Check 4, then 7.
- 1.
- For small/unsorted data or one-off searches.
- It halves the remaining search space.
- 1 or None.
- Sorting itself takes time and may not be worth it for one search.

Independent Practice

- Write three short code examples using Searching Algorithms.
- Change one example so it handles a different input or condition.
- Write one deliberately incorrect example, then correct it and explain the error.
- Use the concept in a short real-life programming task.

Exam Strategy

- Read the complete problem before choosing a structure.
- Trace small examples by hand when output is uncertain.
- When two solutions work, explain differences in clarity, efficiency, or robustness.

Notes & Corrections

Record one mistake you made, the correction, and the programming rule or idea.

LESSON 24 - CORE PROGRAMMING

Sorting Algorithms

Concepts, syntax, and model examples

Learning Objectives		
- Explain bubble, selection, and insertion sort at a basic level. - Trace sorting passes on small lists. - Compare simple sorting strategies.		
Concept	Explanation	Example
Bubble sort	Repeatedly swaps adjacent out-of-order items.	Largest values move toward the end.
Selection sort	Selects the smallest remaining value for the next position.	Find minimum and swap.
Insertion sort	Inserts each new item into the correct place in a sorted prefix.	Similar to arranging cards.
Built-in sort	Uses optimized library sorting in real programs.	<code>items.sort()</code> / <code>sorted(items)</code>

Key Rules

- Trace with a short list to understand the algorithm.
- Simple quadratic sorts are educational but slow on large data.
- Prefer tested built-in sorting unless implementing algorithms for learning or special needs.

Study Tip

Type each model example yourself. Change one value or condition and predict the result before running it.

Extra Model Code / Sentences

```
1. values = [5, 2, 4]
2. Bubble pass: [2,5,4] -> [2,4,5]
3. Next pass confirms order
4. sorted(values) -> [2,4,5]
```

Worked Analysis Routine

Step 1: Identify the inputs, outputs, and target operation.

Step 2: Trace variables and conditions in execution order.

Step 3: Explain why the chosen syntax or structure matches the intended logic.

Step 4: Modify one input or rule and predict how the output changes.

Your Example

Write one accurate example or short code fragment and explain the target programming idea.

LESSON 24 - PRACTICE

Sorting Algorithms

Common errors, guided practice, and mastery check

Common error	Better form	Why?
Assume one bubble pass always fully sorts	Repeat passes until no swaps or enough passes.	One pass may not finish sorting.
Reimplement sorting in production without reason	Use built-in sorting.	Built-ins are optimized and tested.

Guided Practice

- Sort [3,1,2] using bubble steps.
- Which sort inserts into a sorted prefix?
- Which sort repeatedly selects a minimum?
- What does sorted(list) return?
- What does list.sort() do?
- Why study simple sorts?
- Why avoid them for huge lists?
- State one measure used to compare algorithms.

Mastery Challenge

- Write two original examples demonstrating the main concept of this lesson.
- Trace or explain the expected output/result in one short paragraph.
- Check syntax, indentation, names, and edge cases before comparing with model answers.

Quick Answers

- [1,3,2] -> [1,2,3].
- Insertion sort.
- Selection sort.
- A new sorted list.
- Sorts the list in place.
- To learn algorithmic thinking and complexity.
- They often take quadratic time.
- Time or space complexity.

Independent Practice

- Write three short code examples using Sorting Algorithms.
- Change one example so it handles a different input or condition.
- Write one deliberately incorrect example, then correct it and explain the error.
- Use the concept in a short real-life programming task.

Exam Strategy

- Read the complete problem before choosing a structure.
- Trace small examples by hand when output is uncertain.
- When two solutions work, explain differences in clarity, efficiency, or robustness.

Notes & Corrections

Record one mistake you made, the correction, and the programming rule or idea.

LESSON 25 - CORE PROGRAMMING

Algorithm Complexity and Efficiency

Concepts, syntax, and model examples

Learning Objectives

- Interpret basic Big-O notation.
- Compare constant, linear, and quadratic growth.
- Consider time-space trade-offs.

Concept	Explanation	Example
O(1)	Work stays roughly constant as input grows.	Access list item by index.
O(n)	Work grows proportionally with input size.	Linear search.
O(log n)	Work grows slowly by repeated halving.	Binary search.
O(n^2)	Work grows roughly with square of input.	Simple nested pair comparisons.

Key Rules

- Big-O describes growth, not exact execution time.
- Focus on dominant growth for large inputs.
- Performance choices should consider real constraints, readability, and data size.

Study Tip

Type each model example yourself. Change one value or condition and predict the result before running it.

Extra Model Code / Sentences

1. List index access -> O(1)
2. Linear scan -> O(n)
3. Binary search on sorted list -> O(log n)
4. Nested full loops -> often O(n^2)

Worked Analysis Routine

- Step 1: Identify the inputs, outputs, and target operation.
- Step 2: Trace variables and conditions in execution order.
- Step 3: Explain why the chosen syntax or structure matches the intended logic.
- Step 4: Modify one input or rule and predict how the output changes.

Your Example

Write one accurate example or short code fragment and explain the target programming idea.

LESSON 25 - PRACTICE

Algorithm Complexity and Efficiency

Common errors, guided practice, and mastery check

Common error	Better form	Why?
Treat O(n) as exact n seconds	Big-O describes growth, not a measured duration.	Hardware and constants still matter.
Optimize tiny code before correctness	Make it correct and clear first, then measure.	Premature optimization can add complexity.

Guided Practice

1. What does O(1) mean?

2. Give an O(n) example.

3. Give an O(log n) example.

4. What is common for two full nested loops?

5. Which grows faster: n or n^2?

6. Does Big-O give exact time?

7. Why measure before optimizing?

8. Give one time-space trade-off example.

Mastery Challenge

- Write two original examples demonstrating the main concept of this lesson.
- Trace or explain the expected output/result in one short paragraph.
- Check syntax, indentation, names, and edge cases before comparing with model answers.

Quick Answers

1. Roughly constant work with input size.

2. Linear search.

3. Binary search.

4. O(n^2).

5. n^2.

6. No.

7. To optimize real bottlenecks rather than guesses.

8. Caching uses more memory to save repeated computation.

Independent Practice

- A. Write three short code examples using Algorithm Complexity and Efficiency.
- B. Change one example so it handles a different input or condition.
- C. Write one deliberately incorrect example, then correct it and explain the error.
- D. Use the concept in a short real-life programming task.

Exam Strategy

- Read the complete problem before choosing a structure.
- Trace small examples by hand when output is uncertain.
- When two solutions work, explain differences in clarity, efficiency, or robustness.

Notes & Corrections

Record one mistake you made, the correction, and the programming rule or idea.

LESSON 26 - CORE PROGRAMMING

Boolean Logic and Binary Numbers

Concepts, syntax, and model examples

Learning Objectives

- Use Boolean truth values and logic gates conceptually.
- Convert small binary values to decimal and back.
- Relate bit operations to computer data.

Concept	Explanation	Example
Boolean	A value that is True or False.	passed = score >= 50
AND	True only if both inputs are true.	A and B
OR	True if at least one input is true.	A or B
Binary	Base-2 number system using 0 and 1.	1011■ = 11■■■

Key Rules

- Evaluate not before and before or in Python precedence, or use parentheses for clarity.
- Each binary digit represents a power of 2.
- Boolean expressions are central to conditions and loops.

Study Tip

Type each model example yourself. Change one value or condition and predict the result before running it.

Extra Model Code / Sentences

1. 1010■ = 8 + 2 = 10
2. 13■■ = 1101■
3. True and False -> False
4. True or False -> True

Worked Analysis Routine

- Step 1: Identify the inputs, outputs, and target operation.
- Step 2: Trace variables and conditions in execution order.
- Step 3: Explain why the chosen syntax or structure matches the intended logic.
- Step 4: Modify one input or rule and predict how the output changes.

Your Example

Write one accurate example or short code fragment and explain the target programming idea.

LESSON 26 - PRACTICE

Boolean Logic and Binary Numbers

Common errors, guided practice, and mastery check

Common error	Better form	Why?
Assume 1010 binary equals decimal 1010	Interpret using powers of 2.	Binary uses base 2.
Write complex logic without parentheses	Group expressions clearly.	This avoids precedence confusion.

Guided Practice

1. Convert 1011 to decimal.

2. Convert 10 decimal to binary.

3. Evaluate True and True.

4. Evaluate True and False.

5. Evaluate True or False.

6. Evaluate not True.

7. What powers are represented by 4 binary bits?

8. Give one programming use of Boolean logic.

Mastery Challenge

- Write two original examples demonstrating the main concept of this lesson.
- Trace or explain the expected output/result in one short paragraph.
- Check syntax, indentation, names, and edge cases before comparing with model answers.

Quick Answers

1. 5.

2. 101011.

3. True.

4. False.

5. True.

6. False.

7. 8,4,2,1.

8. Conditions, validation, access control, loops, etc.

Independent Practice

- A. Write three short code examples using Boolean Logic and Binary Numbers.
- B. Change one example so it handles a different input or condition.
- C. Write one deliberately incorrect example, then correct it and explain the error.
- D. Use the concept in a short real-life programming task.

Exam Strategy

- Read the complete problem before choosing a structure.
- Trace small examples by hand when output is uncertain.
- When two solutions work, explain differences in clarity, efficiency, or robustness.

Notes & Corrections

Record one mistake you made, the correction, and the programming rule or idea.

LESSON 27 - CORE PROGRAMMING

Data Representation and Memory

Concepts, syntax, and model examples

Learning Objectives

- Explain bits, bytes, and basic data encoding.
- Relate variable types to stored representations conceptually.
- Estimate simple storage sizes.

Concept	Explanation	Example
Bit	Smallest binary unit: 0 or 1.	One Boolean possibility bit conceptually.
Byte	Usually 8 bits.	1 byte = 8 bits
Text encoding	Maps characters to numeric codes.	ASCII/Unicode
Memory	Stores program instructions and data during execution.	Variables occupy memory while a program runs.

Key Rules

- Do not assume one character always uses exactly one byte in every encoding.
- Distinguish storage size from displayed length.
- Understand representation limits when choosing numeric types in lower-level languages.

Study Tip

Type each model example yourself. Change one value or condition and predict the result before running it.

Extra Model Code / Sentences

1. 8 bits = 1 byte
2. 1024 bytes is often called 1 KiB
3. Text characters are stored using numeric code points
4. Images are represented by pixel values and metadata

Worked Analysis Routine

- Step 1: Identify the inputs, outputs, and target operation.
- Step 2: Trace variables and conditions in execution order.
- Step 3: Explain why the chosen syntax or structure matches the intended logic.
- Step 4: Modify one input or rule and predict how the output changes.

Your Example

Write one accurate example or short code fragment and explain the target programming idea.

LESSON 27 - PRACTICE

Data Representation and Memory

Common errors, guided practice, and mastery check

Common error	Better form	Why?
Assume all text characters use one byte	Unicode may use multiple bytes in encodings such as UTF-8.	Encoding affects storage.
Confuse bit with byte	1 byte = 8 bits.	They are different units.

Guided Practice

- How many bits in one byte?
- What is a bit?
- What does text encoding do?
- Name one text encoding family.
- Why can an emoji use more storage than an ASCII letter?
- What is RAM used for conceptually?
- Convert 16 bytes to bits.
- Explain why data representation matters in programming.

Mastery Challenge

- Write two original examples demonstrating the main concept of this lesson.
- Trace or explain the expected output/result in one short paragraph.
- Check syntax, indentation, names, and edge cases before comparing with model answers.

Quick Answers

- 8.
- A binary 0/1 unit.
- Maps characters to numeric representations.
- Unicode/UTF-8, ASCII, etc.
- Its encoded representation may use more bytes.
- Temporary working storage for running programs/data.
- 128 bits.
- It affects correctness, storage, limits, and interoperability.

Independent Practice

- A. Write three short code examples using Data Representation and Memory.
- B. Change one example so it handles a different input or condition.
- C. Write one deliberately incorrect example, then correct it and explain the error.
- D. Use the concept in a short real-life programming task.

Exam Strategy

- Read the complete problem before choosing a structure.
- Trace small examples by hand when output is uncertain.
- When two solutions work, explain differences in clarity, efficiency, or robustness.

Notes & Corrections

Record one mistake you made, the correction, and the programming rule or idea.

LESSON 28 - CORE PROGRAMMING

Stacks and Queues

Concepts, syntax, and model examples

Learning Objectives

- Explain stack and queue behavior.
- Use LIFO and FIFO correctly.
- Recognize common real-world applications.

Concept	Explanation	Example
Stack	Last In, First Out structure.	Push and pop at one end.
Queue	First In, First Out structure.	Enqueue at rear, dequeue at front.
Stack use	Useful for undo, call stacks, parsing.	Browser back history
Queue use	Useful for tasks waiting in order.	Printer jobs, customer requests

Key Rules

- A stack removes the most recently added item.
- A queue removes the earliest waiting item.
- Choose a data structure based on required access order.

Study Tip

Type each model example yourself. Change one value or condition and predict the result before running it.

Extra Model Code / Sentences

1. Stack: push A, B, C -> pop C first
2. Queue: enqueue A, B, C -> dequeue A first
3. Python list can model a simple stack
4. collections.deque is efficient for queues

Worked Analysis Routine

- Step 1: Identify the inputs, outputs, and target operation.
- Step 2: Trace variables and conditions in execution order.
- Step 3: Explain why the chosen syntax or structure matches the intended logic.
- Step 4: Modify one input or rule and predict how the output changes.

Your Example

Write one accurate example or short code fragment and explain the target programming idea.

LESSON 28 - PRACTICE

Stacks and Queues

Common errors, guided practice, and mastery check

Common error	Better form	Why?
Describe queue as LIFO	Queue is FIFO.	First arrival leaves first.
Remove first item from a large list repeatedly	Use deque for efficient queue operations.	Front removal from lists can be costly.

Guided Practice

1. What does LIFO mean?
2. What does FIFO mean?
3. If stack gets A,B,C, what pops first?
4. If queue gets A,B,C, what leaves first?
5. Name one stack application.
6. Name one queue application.
7. Which structure models undo?
8. Which structure models a checkout line?

Mastery Challenge

- Write two original examples demonstrating the main concept of this lesson.
- Trace or explain the expected output/result in one short paragraph.
- Check syntax, indentation, names, and edge cases before comparing with model answers.

Quick Answers

1. Last In, First Out.
2. First In, First Out.
3. C.
4. A.
5. Undo, call stack, backtracking, etc.
6. Printer jobs, task scheduling, service line, etc.
7. Stack.
8. Queue.

Independent Practice

- A. Write three short code examples using Stacks and Queues.
- B. Change one example so it handles a different input or condition.
- C. Write one deliberately incorrect example, then correct it and explain the error.
- D. Use the concept in a short real-life programming task.

Exam Strategy

- Read the complete problem before choosing a structure.
- Trace small examples by hand when output is uncertain.
- When two solutions work, explain differences in clarity, efficiency, or robustness.

Notes & Corrections

Record one mistake you made, the correction, and the programming rule or idea.

LESSON 29 - CORE PROGRAMMING

Databases and SQL Basics

Concepts, syntax, and model examples

Learning Objectives

- Explain tables, rows, columns, and keys.
- Write basic SELECT, INSERT, UPDATE, and DELETE statements conceptually.
- Connect programming logic with persistent data.

Concept	Explanation	Example
Table	Stores records with named columns.	Students(id, name, score)
Primary key	Uniquely identifies a row.	id
SELECT	Reads data from a table.	SELECT name FROM Students;
WHERE	Filters rows by a condition.	WHERE score >= 50

Key Rules

- Use primary keys to identify records reliably.
- Never build SQL from untrusted input by string concatenation in real applications; use parameters.
- Back up data and test destructive UPDATE/DELETE statements carefully.

Study Tip

Type each model example yourself. Change one value or condition and predict the result before running it.

Extra Model Code / Sentences

1. SELECT name, score FROM Students;
2. SELECT * FROM Students WHERE score >= 50;
3. INSERT INTO Students(name,score) VALUES (?,?);
4. UPDATE Students SET score=? WHERE id=?;

Worked Analysis Routine

- Step 1: Identify the inputs, outputs, and target operation.
- Step 2: Trace variables and conditions in execution order.
- Step 3: Explain why the chosen syntax or structure matches the intended logic.
- Step 4: Modify one input or rule and predict how the output changes.

Your Example

Write one accurate example or short code fragment and explain the target programming idea.

LESSON 29 - PRACTICE

Databases and SQL Basics

Common errors, guided practice, and mastery check

Common error	Better form	Why?
DELETE FROM Students; without intending all rows	Use WHERE to target records.	Without WHERE, all rows may be deleted.
Concatenate user input directly into SQL	Use parameterized queries.	Parameters reduce injection risk and quoting errors.

Guided Practice

1. What is a database table?

2. What is a primary key?

3. Write `SELECT *` from Students.

4. Filter score `>= 50`.

5. Why use WHERE with UPDATE/DELETE?

6. What does INSERT do?

7. Why parameterize SQL?

8. Name two persistent data use cases.

Mastery Challenge

- Write two original examples demonstrating the main concept of this lesson.
- Trace or explain the expected output/result in one short paragraph.
- Check syntax, indentation, names, and edge cases before comparing with model answers.

Quick Answers

1. A structured collection of rows and columns.

2. A unique row identifier.

3. `SELECT * FROM Students;`

4. `SELECT * FROM Students WHERE score >= 50;`

5. To target intended records.

6. Adds a new row.

7. Security and correctness.

8. Student records, inventory, accounts, logs, etc.

Independent Practice

- A. Write three short code examples using Databases and SQL Basics.
- B. Change one example so it handles a different input or condition.
- C. Write one deliberately incorrect example, then correct it and explain the error.
- D. Use the concept in a short real-life programming task.

Exam Strategy

- Read the complete problem before choosing a structure.
- Trace small examples by hand when output is uncertain.
- When two solutions work, explain differences in clarity, efficiency, or robustness.

Notes & Corrections

Record one mistake you made, the correction, and the programming rule or idea.

LESSON 30 - CORE PROGRAMMING

Mini-Project Design and Programming Review

Concepts, syntax, and model examples

Learning Objectives

- Plan a small complete program from requirements to tests.
- Combine input, decisions, loops, functions, and data structures.
- Document and evaluate a finished solution.

Concept	Explanation	Example
Requirements	State what the program must do.	Inputs, outputs, rules, constraints
Design	Plan algorithms, functions, and data structures.	Pseudocode + flow + module list
Implementation	Write and integrate code incrementally.	Build one function at a time
Evaluation	Test against requirements and identify improvements.	Correctness, usability, efficiency, maintainability

Key Rules

- Start with a small working version before adding features.
- Keep functions focused and test them independently.
- Document assumptions, input rules, and known limitations.

Study Tip

Type each model example yourself. Change one value or condition and predict the result before running it.

Extra Model Code / Sentences

1. Project: Student Grade Analyzer
2. Input a list of marks
3. Compute average, min, max, pass count
4. Display formatted report and save results

Worked Analysis Routine

Step 1: Identify the inputs, outputs, and target operation.
 Step 2: Trace variables and conditions in execution order.
 Step 3: Explain why the chosen syntax or structure matches the intended logic.
 Step 4: Modify one input or rule and predict how the output changes.

Your Example

Write one accurate example or short code fragment and explain the target programming idea.

LESSON 30 - PRACTICE

Mini-Project Design and Programming Review

Common errors, guided practice, and mastery check

Common error	Better form	Why?
Attempt every feature at once	Build a minimal version, test, then extend.	Incremental development reduces debugging complexity.
No test plan	Define normal, boundary, and invalid cases.	A project is incomplete without verification.

Guided Practice

- Write three requirements for a grade analyzer.
- List inputs and outputs.
- Name three functions you would create.
- Choose a data structure for marks.
- Give two test cases.
- State one usability improvement.
- State one efficiency improvement.
- Write a short evaluation criterion.

Mastery Challenge

- Write two original examples demonstrating the main concept of this lesson.
- Trace or explain the expected output/result in one short paragraph.
- Check syntax, indentation, names, and edge cases before comparing with model answers.

Quick Answers

- Answers vary.
- Input marks; output statistics/report.
- read_marks, calculate_stats, display_report, etc.
- List.
- Normal and boundary/invalid examples.
- Clear prompts/error messages.
- Avoid repeated calculations; use efficient structures.
- Correctness, clarity, speed, maintainability, etc.

Independent Practice

- A. Write three short code examples using Mini-Project Design and Programming Review.
- B. Change one example so it handles a different input or condition.
- C. Write one deliberately incorrect example, then correct it and explain the error.
- D. Use the concept in a short real-life programming task.

Exam Strategy

- Read the complete problem before choosing a structure.
- Trace small examples by hand when output is uncertain.
- When two solutions work, explain differences in clarity, efficiency, or robustness.

Notes & Corrections

Record one mistake you made, the correction, and the programming rule or idea.

REVISION 01

Review Test 1 - Foundations & Design

Mixed practice - complete without looking back at the lessons.

#	Lesson	Question
1	Programming Fundamentals and Problem Solving	Define programming in one sentence.
2	Programming Fundamentals and Problem Solving	List the six main stages of the problem-solving cycle.
3	Algorithms and Pseudocode	Write pseudocode to add three numbers.
4	Algorithms and Pseudocode	Write pseudocode to test whether a number is positive.
5	Flowcharts and Program Design	Name the symbol used for a process.
6	Flowcharts and Program Design	Draw or describe a flowchart for checking voting age.
7	Variables, Constants and Data Types	Classify 42, 3.5, "Hello", True by type.
8	Variables, Constants and Data Types	Explain the difference between integer and float.
9	Operators and Expressions	Evaluate $2 + 3 * 4$.
10	Operators and Expressions	What does $17 \% 5$ return?
11	Operators and Expressions	Explain <code>==</code> versus <code>=</code> .
12	Variables, Constants and Data Types	Why is "100" different from 100?

Writing / Coding Task

Write a short program or pseudocode solution using at least four programming ideas reviewed on this page. Add comments identifying the four target concepts.

Self-Assessment

- Score 10-12: Strong mastery.
- Score 7-9: Good progress; review minor errors.
- Score 4-6: Revisit the related lessons.
- Score 0-3: Study the lesson explanations before retrying.

Coding Task Response / Notes

Plan or write your solution here. Use at least four target programming points.

REVISION 02

Review Test 2 - Input, Selection & Loops

Mixed practice - complete without looking back at the lessons.

#	Lesson	Question
1	Input, Output and Type Conversion	Write input for a student name.
2	Input, Output and Type Conversion	Read an integer quantity.
3	Input, Output and Type Conversion	Why does input() need conversion for arithmetic?
4	Conditional Statements	Write an if statement for adult age.
5	Conditional Statements	Use and to test a range.
6	Conditional Statements	Write a condition for n not equal to zero.
7	While Loops	Write a while loop printing 1 to 3.
8	While Loops	What causes an infinite loop?
9	For Loops and Range	Write a loop from 1 to 10.
10	For Loops and Range	Write a countdown 5 to 1.
11	For Loops and Range	Iterate over letters in "CODE".
12	For Loops and Range	Explain the stop parameter.

Writing / Coding Task

Write a short program or pseudocode solution using at least four programming ideas reviewed on this page. Add comments identifying the four target concepts.

Self-Assessment

- Score 10-12: Strong mastery.
- Score 7-9: Good progress; review minor errors.
- Score 4-6: Revisit the related lessons.
- Score 0-3: Study the lesson explanations before retrying.

Coding Task Response / Notes

Plan or write your solution here. Use at least four target programming points.

REVISION 03

Review Test 3 - Text, Collections & Functions

Mixed practice - complete without looking back at the lessons.

#	Lesson	Question
1	Strings and Text Processing	What is "CODE"[0]?
2	Strings and Text Processing	Slice "Python" to get "Pyt".
3	Strings and Text Processing	Count the length of a string.
4	Lists and Arrays	Access the second item.
5	Lists and Arrays	Append one item.
6	Tuples, Sets and Dictionaries	Create a dictionary for name and age.
7	Tuples, Sets and Dictionaries	Which collection is immutable?
8	Functions	Define square(x).
9	Functions	Explain parameter versus argument.
10	Functions	What happens after return?
11	Parameters, Return Values and Scope	What is local scope?
12	Parameters, Return Values and Scope	Explain why globals can be risky.

Writing / Coding Task

Write a short program or pseudocode solution using at least four programming ideas reviewed on this page. Add comments identifying the four target concepts.

Self-Assessment

- Score 10-12: Strong mastery.
- Score 7-9: Good progress; review minor errors.
- Score 4-6: Revisit the related lessons.
- Score 0-3: Study the lesson explanations before retrying.

Coding Task Response / Notes

Plan or write your solution here. Use at least four target programming points.

REVISION 04

Review Test 4 - Scope, Recursion, Modules & Files

Mixed practice - complete without looking back at the lessons.

#	Lesson	Question
1	Parameters, Return Values and Scope	What is a default parameter?
2	Parameters, Return Values and Scope	Rewrite a global dependency using a parameter.
3	Recursion	Define recursion.
4	Recursion	Trace factorial(3).
5	Recursion	Why can recursion cause a stack overflow?
6	Modules and Libraries	Import math.
7	Modules and Libraries	Use math.pi.
8	Modules and Libraries	Explain namespace.
9	File Handling	Open notes.txt for reading.
10	File Handling	Append to log.txt.
11	File Handling	Explain with open.
12	File Handling	Why can w be dangerous?

Writing / Coding Task

Write a short program or pseudocode solution using at least four programming ideas reviewed on this page. Add comments identifying the four target concepts.

Self-Assessment

- Score 10-12: Strong mastery.
- Score 7-9: Good progress; review minor errors.
- Score 4-6: Revisit the related lessons.
- Score 0-3: Study the lesson explanations before retrying.

Coding Task Response / Notes

Plan or write your solution here. Use at least four target programming points.

REVISION 05

Review Test 5 - Debugging, Testing & OOP

Mixed practice - complete without looking back at the lessons.

#	Lesson	Question
1	Exceptions and Debugging	Classify a missing colon error.
2	Exceptions and Debugging	Classify division by zero.
3	Exceptions and Debugging	Classify wrong formula.
4	Exceptions and Debugging	What does try/except do?
5	Exceptions and Debugging	Why inspect a traceback?
6	Testing and Test Cases	Give a normal test for 0-100 score.
7	Testing and Test Cases	Give two boundary tests.
8	Testing and Test Cases	What is an assertion?
9	Testing and Test Cases	Design tests for password length ≥ 8 .
10	Object-Oriented Programming Concepts	Define a class in words.
11	Object-Oriented Programming Concepts	Define an object.
12	Object-Oriented Programming Concepts	What does self refer to?

Writing / Coding Task

Write a short program or pseudocode solution using at least four programming ideas reviewed on this page. Add comments identifying the four target concepts.

Self-Assessment

- Score 10-12: Strong mastery.
- Score 7-9: Good progress; review minor errors.
- Score 4-6: Revisit the related lessons.
- Score 0-3: Study the lesson explanations before retrying.

Coding Task Response / Notes

Plan or write your solution here. Use at least four target programming points.

REVISION 06

Review Test 6 - Classes, Inheritance & Searching

Mixed practice - complete without looking back at the lessons.

#	Lesson	Question
1	Classes and Objects in Python	Write a class Book with title.
2	Classes and Objects in Python	Add a method display_title.
3	Classes and Objects in Python	What does __init__ do?
4	Classes and Objects in Python	What is instantiation?
5	Inheritance and Polymorphism	What is a subclass?
6	Inheritance and Polymorphism	What is overriding?
7	Inheritance and Polymorphism	Give an is-a example.
8	Inheritance and Polymorphism	Explain polymorphism simply.
9	Searching Algorithms	Describe linear search.
10	Searching Algorithms	What is required for binary search?
11	Searching Algorithms	What index should be returned?
12	Searching Algorithms	Why is binary search faster on large sorted data?

Writing / Coding Task

Write a short program or pseudocode solution using at least four programming ideas reviewed on this page. Add comments identifying the four target concepts.

Self-Assessment

- Score 10-12: Strong mastery.
- Score 7-9: Good progress; review minor errors.
- Score 4-6: Revisit the related lessons.
- Score 0-3: Study the lesson explanations before retrying.

Coding Task Response / Notes

Plan or write your solution here. Use at least four target programming points.

REVISION 07

Review Test 7 - Sorting, Complexity & Representation

Mixed practice - complete without looking back at the lessons.

#	Lesson	Question
1	Sorting Algorithms	Sort [3,1,2] using bubble steps.
2	Sorting Algorithms	Which sort inserts into a sorted prefix?
3	Sorting Algorithms	What does sorted(list) return?
4	Sorting Algorithms	Why avoid them for huge lists?
5	Algorithm Complexity and Efficiency	What does $O(1)$ mean?
6	Algorithm Complexity and Efficiency	Give an $O(n)$ example.
7	Algorithm Complexity and Efficiency	What is common for two full nested loops?
8	Algorithm Complexity and Efficiency	Does Big-O give exact time?
9	Boolean Logic and Binary Numbers	Convert 101 to decimal.
10	Boolean Logic and Binary Numbers	Convert 10 decimal to binary.
11	Data Representation and Memory	How many bits in one byte?
12	Data Representation and Memory	Why can an emoji use more storage than an ASCII letter?

Writing / Coding Task

Write a short program or pseudocode solution using at least four programming ideas reviewed on this page. Add comments identifying the four target concepts.

Self-Assessment

- Score 10-12: Strong mastery.
- Score 7-9: Good progress; review minor errors.
- Score 4-6: Revisit the related lessons.
- Score 0-3: Study the lesson explanations before retrying.

Coding Task Response / Notes

Plan or write your solution here. Use at least four target programming points.

REVISION 08

Review Test 8 - Data Structures, SQL & Project

Mixed practice - complete without looking back at the lessons.

#	Lesson	Question
1	Stacks and Queues	What does LIFO mean?
2	Stacks and Queues	What does FIFO mean?
3	Stacks and Queues	If stack gets A,B,C, what pops first?
4	Stacks and Queues	Which structure models undo?
5	Databases and SQL Basics	What is a database table?
6	Databases and SQL Basics	What is a primary key?
7	Databases and SQL Basics	Write SELECT * from Students.
8	Databases and SQL Basics	Why parameterize SQL?
9	Mini-Project Design and Programming Review	Write three requirements for a grade analyzer.
10	Mini-Project Design and Programming Review	List inputs and outputs.
11	Mini-Project Design and Programming Review	State one usability improvement.
12	Mini-Project Design and Programming Review	Write a short evaluation criterion.

Writing / Coding Task

Write a short program or pseudocode solution using at least four programming ideas reviewed on this page. Add comments identifying the four target concepts.

Self-Assessment

- Score 10-12: Strong mastery.
- Score 7-9: Good progress; review minor errors.
- Score 4-6: Revisit the related lessons.
- Score 0-3: Study the lesson explanations before retrying.

Coding Task Response / Notes

Plan or write your solution here. Use at least four target programming points.

ANSWER KEY

Answer Key - Part 1

Use this section after completing the exercises independently.

Review Test 1 - Foundations & Design

1. Writing instructions for a computer to solve problems.	2. Analyse, design, code, test, debug, document/improve.	3. READ a,b,c; total<-a+b+c; PRINT total.	4. IF n>0 THEN PRINT "Positive".
5. Rectangle.	6. START -> READ age -> age>=18? -> outputs -> END.	7. int, float, string, bool.	8. Integer is whole; float may include decimals.
9. 14.	10. 2.	11. == compares; = assigns.	12. One is text; the other is numeric.

Review Test 2 - Input, Selection & Loops

1. name=input("Name: ").	2. quantity=int(input("Quantity: ")).	3. Because input returns a string.	4. if age >= 18: ...
5. Example: 10 <= temp <= 30.	6. n != 0.	7. count=1; while count<=3: print(count); count+=1.	8. A condition that never becomes false.
9. for i in range(1,11): ...	10. range(5,0,-1).	11. for ch in "CODE": ...	12. It is excluded.

Correction Method

- For every wrong answer, write the corrected code or explanation and the rule that explains it.
- If the error is repeated, return to the lesson and complete the Mastery Challenge again.

Error Log

Write the item number, your answer, the correction, and the programming rule.

ANSWER KEY

Answer Key - Part 2

Use this section after completing the exercises independently.

Review Test 3 - Text, Collections & Functions

1. C.	2. "Python"[0:3].	3. len(text).	4. items[1].
5. items.append(x).	6. {"name": "...", "age": ...}.	7. Tuple.	8. def square(x): return x*x.
9. Parameter is in definition; argument is supplied in call.	10. The function ends and passes back a value.	11. Variables defined inside a function are local by default.	12. They create hidden dependencies/side effects.

Review Test 4 - Scope, Recursion, Modules & Files

1. A parameter with a value used when caller omits it.	2. Answers vary.	3. A function solving a problem by calling itself on smaller input.	4. 3*2*1=6.
5. Too many nested calls use stack memory.	6. import math.	7. math.pi.	8. A mapping of names that helps avoid collisions.
9. with open("notes.txt", "r") as f: ...	10. open(..., "a").	11. It manages the resource and closes automatically.	12. It can overwrite the file.

Correction Method

- For every wrong answer, write the corrected code or explanation and the rule that explains it.
- If the error is repeated, return to the lesson and complete the Mastery Challenge again.

Error Log

Write the item number, your answer, the correction, and the programming rule.

ANSWER KEY

Answer Key - Part 3

Use this section after completing the exercises independently.

Review Test 5 - Debugging, Testing & OOP

1. Syntax error.	2. Runtime error/ZeroDivisionError.	3. Logic error.	4. Handles selected runtime exceptions.
5. It shows where and how the failure occurred.	6. Example 65.	7. 0 and 100, plus near decision boundaries.	8. A check that a condition/result is true.
9. Lengths 7,8,9 plus invalid types.	10. A blueprint/type describing objects.	11. An instance of a class.	12. The current instance.

Correction Method

- For every wrong answer, write the corrected code or explanation and the rule that explains it.
- If the error is repeated, return to the lesson and complete the Mastery Challenge again.

Error Log

Write the item number, your answer, the correction, and the programming rule.

ANSWER KEY

Answer Key - Part 4

Use this section after completing the exercises independently.

Review Test 6 - Classes, Inheritance & Searching

1. class Book: def __init__(self,title): self.title=title.	2. def display_title(self): print(self.title).	3. Initializes a new object.	4. Creating an object from a class.
5. A class derived from another class.	6. Replacing/inserting subclass behavior for an inherited method.	7. Dog is an Animal.	8. Same interface, different implementations.
9. Check each item in order.	10. Sorted data.	11. 1.	12. It halves the remaining search space.

Correction Method

- For every wrong answer, write the corrected code or explanation and the rule that explains it.
- If the error is repeated, return to the lesson and complete the Mastery Challenge again.

Error Log

Write the item number, your answer, the correction, and the programming rule.

ANSWER KEY

Answer Key - Part 5

Use this section after completing the exercises independently.

Review Test 7 - Sorting, Complexity & Representation

1. [1,3,2] -> [1,2,3].	2. Insertion sort.	3. A new sorted list.	4. They often take quadratic time.
5. Roughly constant work with input size.	6. Linear search.	7. $O(n^2)$.	8. No.
9. 5.	10. 1010■.	11. 8.	12. Its encoded representation may use more bytes.

Correction Method

- For every wrong answer, write the corrected code or explanation and the rule that explains it.
- If the error is repeated, return to the lesson and complete the Mastery Challenge again.

Error Log

Write the item number, your answer, the correction, and the programming rule.

ANSWER KEY

Answer Key - Part 6

Use this section after completing the exercises independently.

Review Test 8 - Data Structures, SQL & Project			
1. Last In, First Out.	2. First In, First Out.	3. C.	4. Stack.
5. A structured collection of rows and columns.	6. A unique row identifier.	7. SELECT * FROM Students;.	8. Security and correctness.
9. Answers vary.	10. Input marks; output statistics/report.	11. Clear prompts/error messages.	12. Correctness, clarity, speed, maintainability, etc.

Correction Method

- For every wrong answer, write the corrected code or explanation and the rule that explains it.
- If the error is repeated, return to the lesson and complete the Mastery Challenge again.

Error Log

Write the item number, your answer, the correction, and the programming rule.

REFERENCE

Python & Programming Reference

Common syntax, operators, functions, and structures

Category	Syntax / Example	Meaning
Assignment	<code>x = 5</code>	Store a value
Arithmetic	<code>+ - * / // % **</code>	Math operators
Comparison	<code>== != < <= > >=</code>	Return True/False
Logic	<code>and or not</code>	Combine Boolean expressions
Selection	<code>if / elif / else</code>	Choose a branch
While loop	<code>while condition:</code>	Repeat while condition is true
For loop	<code>for x in sequence:</code>	Iterate over values
Range	<code>range(start, stop, step)</code>	Generate integer sequence; stop excluded
Function	<code>def name(params): ... return value</code>	Reusable block
List	<code>items = [1,2,3]</code>	Ordered mutable collection
Dictionary	<code>d = {"key": value}</code>	Key-value mapping
Set	<code>s = {1,2,3}</code>	Unique values
File	<code>with open(path, mode) as f:</code>	Open resource safely
Exception	<code>try: ... except ValueError:</code>	Handle selected runtime errors

Useful Built-ins

- `len(x)`: number of items
- `sum(nums)`: sum numeric values
- `min(x), max(x)`: extremes
- `sorted(x)`: new sorted list
- `type(x)`: inspect type
- `int(), float(), str(), bool()`: conversions
- `enumerate(seq)`: index + value pairs
- `range(...)`: integer sequence for loops

Complexity Quick Guide

- $O(1)$: constant-time operation
- $O(\log n)$: repeatedly halve the problem
- $O(n)$: one full scan
- $O(n \log n)$: efficient comparison sorting class
- $O(n^2)$: nested pairwise work on n items
- Big-O describes growth, not exact seconds.

REFERENCE

Programming Quick Guide

Final checklist, glossary, and exam reminders

Program Design Checklist

- Define the problem, inputs, outputs, and constraints.
- Write pseudocode or a flowchart before coding complex logic.
- Choose data structures and functions that match the task.
- Build a small working version, then extend it.

Testing Checklist

- Test normal input.
- Test boundary values where conditions change.
- Test invalid input when relevant.
- Write expected results before running the test.
- Retest after fixing a bug.

Debugging Routine

- Read the full error message or traceback.
- Reproduce the error with a small input.
- Inspect variables and conditions near the failure.
- Change one thing at a time and retest.
- Add a regression test after fixing important bugs.

Core Glossary

- Algorithm: finite ordered solution method.
- Variable: named storage whose value may change.
- Function: reusable block that may take parameters and return a value.
- Loop: structure for repetition.
- Class: blueprint for objects.
- Data structure: organized way to store and access data.
- Complexity: how resource use grows with input size.
- Database: persistent structured data store.

Exam Reminders

- Trace code carefully: assignment changes values immediately.
- Remember Python indexes start at 0 and range stop is excluded.
- Use == for comparison, = for assignment.
- Check loop termination and indentation.
- Explain both what the code does and why the structure is appropriate.